

پرسش و پاسخ‌های کلیدی مدل‌های زبان بزرگ (LLMs)

پرسش ۱: چرا مدل‌های CNN و RNN از جاسازی‌های موقعیتی (positional embeddings) استفاده نمی‌کنند، اما ترنسفورمرها از آن‌ها استفاده می‌کنند؟

مدل‌های CNN و RNN از جاسازی‌های موقعیتی استفاده نمی‌کنند زیرا این مدل‌ها به‌طور ذاتی اطلاعات موقعیتی را از طریق فیلترهای کانولوشنی یا تکرار در طول گام‌های زمانی ثبت می‌کنند. با این حال، مدل ترنسفورمر از مکانیزم خود-توجهی (self-attention) استفاده می‌کند که تمام توکن‌ها را به‌صورت موازی و بدون هیچ درکی از توالی یا ترتیب پردازش می‌کند. جاسازی‌های موقعیتی اطلاعات صریحی درباره موقعیت هر توکن در توالی به مدل تزریق می‌کنند و به ترنسفورمرها اجازه می‌دهند تا ترتیب کلمات و فواصل نسبی بین توکن‌ها را درک کنند. این امر به آن‌ها کمک می‌کند تا وابستگی‌های متوالی و ساختار جمله را به‌طور مؤثر مدل‌سازی کنند و ماهیت موازی و بی‌توجه به ترتیب مکانیزم‌های خود-توجهی را جبران کنند.

پرسش ۲: مراحل اصلی در اجرای یک پرس‌وجوی استنتاجی (inference query) بر روی یک LLM را بیان کنید.

اجرای یک پرس‌وجوی استنتاجی بر روی یک LLM شامل مراحل زیر است:

- **توکن‌سازی (Tokenization):** ابتدا متن ورودی به توکن‌ها تقسیم می‌شود و سپس توکن‌ها به شناسه‌های خود در واژگان مدل نگاشت می‌شوند.
- **فاز پیش‌پر کردن (Prefill Phase):** لایه جاسازی (embedding) این شناسه‌های توکن را به جاسازی‌ها تبدیل می‌کند و سپس لایه‌های رمزگشا (decoder) این جاسازی‌ها را به‌طور همزمان پردازش می‌کنند تا حالت‌های میانی (کلیدها و مقادیر) را محاسبه کنند. این پردازش موازی، زمینه را برای تولید توکن‌های جدید فراهم می‌کند.
- **فاز رمزگشایی (Decoding Phase):** مدل توکن‌های خروجی را به‌صورت خودبازگشتی (autoregressively) و یکی‌یکی تولید می‌کند، که هر کدام بر اساس توکن‌های تولید شده قبلی و حالت‌های ذخیره شده (cached states) است و این فرآیند تا رسیدن به شرط توقف ادامه می‌یابد.
- **بازگردانی توکن‌ها (Detokenization):** در نهایت، توکن‌های تولید شده دوباره به متن قابل خواندن برای انسان تبدیل می‌شوند.

پرسش ۳: توضیح دهید که چگونه KV Cache استنتاج LLM را تسریع می‌کند.

KV Cache با ذخیره کردن نمایش‌های کلید (K) و مقدار (V) توجه که برای توکن‌های قبلی محاسبه شده‌اند، سرعت استنتاج LLM را افزایش می‌دهد. این کار به مدل اجازه می‌دهد تا به‌جای محاسبه مجدد آن‌ها در هر مرحله رمزگشایی، از آن‌ها دوباره استفاده کند. در طول تولید متن، تنها پرس‌وجو (query) برای آخرین توکن محاسبه شده و سپس با بردارهای K و V ذخیره شده ترکیب می‌شود تا توکن بعدی تولید شود، که این امر به‌طور قابل توجهی محاسبات اضافی را کاهش می‌دهد. این فرآیند به‌طرز چشمگیری سرعت استنتاج را افزایش می‌دهد—اغلب منجر به تولید توکن چندین برابر سریع‌تر می‌شود. با این حال، این حافظه پنهان (cache) مقدار قابل توجهی از حافظه GPU را مصرف می‌کند، به همین دلیل از تکنیک‌هایی مانند آفلود کردن و فشرده‌سازی KV cache برای ایجاد تعادل بین سرعت و کارایی حافظه در سرویس‌دهی LLM در مقیاس بزرگ استفاده می‌شود.

پرسش ۴: کوانتیزه‌سازی (quantization) چگونه بر سرعت استنتاج و نیازمندی‌های حافظه تأثیر می‌گذارد؟

کوانتیزه‌سازی دقت عددی پارامترهای LLM را کاهش می‌دهد، مانند تبدیل وزن‌های ممیز شناور ۳۲ بیتی به عرض بیت‌های پایین‌تر مانند اعداد صحیح ۸ بیتی یا مقادیر ۴ بیتی. این فشردگی به‌طور چشمگیری ردپای حافظه مدل را کاهش می‌دهد، اغلب تا ۵۰-۷۵٪ یا بیشتر، که امکان استقرار بر روی دستگاه‌های کوچکتر را فراهم کرده و اجازه می‌دهد داده‌های بیشتری در حافظه پنهان (cache) جای گیرند. همزمان، کوانتیزه‌سازی سرعت استنتاج را افزایش می‌دهد زیرا عملیات با دقت پایین‌تر به منابع محاسباتی کمتری نیاز دارند و می‌توانند توسط سخت‌افزارهای مدرن که برای چنین فرمت‌هایی بهینه‌سازی شده‌اند، سریع‌تر پردازش شوند. با این حال، این افزایش کارایی ممکن است با هزینه کوچکی در دقت مدل همراه باشد، که نیازمند کالیبراسیون یا آموزش آگاه از کوانتیزه‌سازی (quantization-aware training) برای به حداقل رساندن افت عملکرد است. در مجموع، کوانتیزه‌سازی یک تکنیک حیاتی برای کاهش مصرف حافظه و بهبود سرعت استنتاج در کاربردهای عملی LLM است.

پرسش ۵: چگونه با نیازمندی‌های بالای حافظه KV cache در استنتاج LLM مقابله می‌کنید؟

مدیریت نیازمندی‌های بالای حافظه KV Cache در استنتاج LLM برای سرویس‌دهی با توان عملیاتی بالا و پنجره‌های زمینه طولانی بسیار حیاتی است، زیرا اندازه حافظه پنهان به‌صورت خطی با طول توالی و اندازه دسته (batch size) افزایش می‌یابد. ما می‌توانیم با استفاده از تکنیک‌هایی مانند PagedAttention، Grouped-Query Attention، Multi-Query Attention، کوانتیزه‌سازی، یا آفلود کردن کش (Cache Offloading) با نیازمندی‌های بالای حافظه KV Cache مقابله کنیم. PagedAttention با استفاده از بلوک‌های با اندازه ثابت، مانند حافظه مجازی یک سیستم عامل، کش را مدیریت می‌کند، پراکندگی حافظه را کاهش می‌دهد و امکان استفاده از دسته‌های بزرگتر را فراهم می‌کند. Grouped-Query Attention (GQA) یا Multi-Query Attention (MQA) با کاهش تعداد هدهای کلید/مقدار (Key/Value)، اندازه KV cache را کاهش می‌دهند. در نهایت، استراتژی‌هایی مانند کوانتیزه‌سازی KV cache (مثلاً به INT8) یا آفلود کردن داده‌های غیرفعال کش به حافظه‌های ارزان‌تر (مانند رم CPU) نیز به مدیریت مؤثر ردپای حافظه کمک می‌کنند.

پرسش ۶: پس از توکن‌سازی، توکن‌ها چگونه در مدل ترنسفورمر به جاسازی‌ها (embeddings) تبدیل می‌شوند؟

توکن‌ها که پس از توکن‌سازی به‌عنوان شناسه‌های عددی صحیح نمایش داده می‌شوند، با استفاده از یک جدول جستجو به نام ماتریس جاسازی (embedding matrix) به جاسازی‌ها تبدیل می‌شوند. برای هر توکن در واژگان مدل، این ماتریس یک بردار متراکم با اندازه ثابت (جاسازی) دارد. به‌طور خاص، شناسه توکن به‌عنوان یک شاخص برای بازیابی بردار با ابعاد بالای مربوط به خود از ماتریس جاسازی استفاده می‌شود. این بردار بازیابی شده، نمایش عددی است که معنای معنایی و نحوی توکن را در بر می‌گیرد. سپس جاسازی‌های توکن‌های ورودی توسط لایه‌های بعدی ترنسفورمر پردازش می‌شوند.

پرسش ۷: توضیح دهید چرا توکن‌سازی زیرکلمه‌ای (subword tokenization) بر توکن‌سازی سطح کلمه (word-level tokenization) در مدل ترنسفورمر ترجیح داده می‌شود.

توکن‌سازی زیرکلمه‌ای، مانند (Byte-Pair Encoding (BPE در مدل ترنسفورمر، با مدیریت کلمات خارج از واژگان و ایجاد تعادل در اندازه واژگان، چندین مزیت نسبت به توکن‌سازی سطح کلمه ارائه می‌دهد. این روش به‌طور مؤثری کلمات خارج از واژگان (OOV) (کلمات دیده‌نشده، یعنی کلماتی که در واژگان مدل نیستند) را با شکستن آن‌ها به زیرکلمات شناخته‌شده مدیریت می‌کند و پوشش کلمات نادر یا دیده‌نشده را به‌طور قابل توجهی بهبود می‌بخشد. این روش همچنین منجر به یک واژگان کوچک‌تر و قابل مدیریت‌تر در مقایسه با لغت‌نامه گسترده توکن‌سازی سطح کلمه می‌شود، در حالی که همچنان مزایای معنایی واحدهای سطح کلمه را حفظ می‌کند. در نهایت، با تقسیم کلمات به بخش‌های معنادار، به مدل کمک می‌کند تا در مورد مورفولوژی (پیشوندها، پسوندها) بیاموزد.

پرسش ۸: مزایا و معایب استفاده از واژگان بزرگ در LLMها را توضیح دهید.

استفاده از واژگان بزرگ در LLMها به مدل اجازه می‌دهد تا کلمات را با دقت بیشتری نمایش دهد و تکه‌تکه شدن توکن‌ها را کاهش دهد، که این امر درک اصطلاحات نادر یا تخصصی را بهبود می‌بخشد. با این حال، این کار اندازه لایه‌های جاسازی و خروجی را افزایش می‌دهد و منجر به مصرف حافظه بیشتر و آموزش کندتر می‌شود. واژگان بزرگ‌تر همچنین محاسبات softmax را گران‌تر می‌کنند و ممکن است منجر به نمایش پراکنده‌تر توکن‌های کمتر رایج شوند، که این امر به‌طور بالقوه مانع از یادگیری مؤثر برای آن کلمات می‌شود. بنابراین، مدل‌ها اغلب اندازه واژگان را تعدیل می‌کنند تا تعادلی بین پوشش زبانی و کارایی محاسباتی برقرار کنند.

پرسش ۹: نحوه محاسبه خود-توجهی (self-attention) در مدل ترنسفورمر را گام به گام توضیح دهید.

خود-توجهی در مدل ترنسفورمر با این مراحل محاسبه می‌شود: ابتدا هر توکن ورودی با استفاده از ماتریس‌های وزن آموخته‌شده به سه بردار پرس‌وجو (queries)، کلید (keys) و مقدار (values) تبدیل می‌شود. سپس، امتیازات توجه با محاسبه ضرب داخلی (dot product) یک پرس‌وجو با تمام کلیدها محاسبه شده و برای پایدارسازی گرادیان‌ها بر جذر ابعاد کلید تقسیم می‌شود. این امتیازات با استفاده از تابع softmax نرمال‌سازی می‌شوند تا وزن‌های توجه به دست آیند که نشان‌دهنده اهمیت هر توکن نسبت به سایرین است. در نهایت، خروجی هر توکن به‌عنوان مجموع وزن‌دار بردارهای مقدار محاسبه می‌شود، که به مدل اجازه می‌دهد روابط متنی درون توالی ورودی را درک کند.

پرسش ۱۰: پیچیدگی محاسباتی خود-توجهی در مدل ترنسفورمر چقدر است؟

پیچیدگی محاسباتی مکانیزم خود-توجهی در مدل ترنسفورمر عمدتاً نسبت به طول توالی n از مرتبه درجه دوم (quadratic) است. به طور رسمی، این پیچیدگی $O(n^2)$ است. عبارت $O(n^2)$ از محاسبه امتیازات توجه بین تمام جفت‌های توکن‌ها ناشی می‌شود. از آنجا که $n \times n = n^2$ تعامل دوتایی وجود دارد و هر کدام شامل یک ضرب داخلی از بردارهای با ابعاد d است، این امر منجر به $O(n^2 d)$ عملیات می‌شود. پیچیدگی درجه دوم نسبت به طول توالی n باعث می‌شود خود-توجهی برای توالی‌های طولانی از نظر محاسباتی پرهزینه باشد. با این حال، این طراحی به مدل اجازه می‌دهد تا روابط بین تمام توکن‌ها را به طور همزمان درک کند، که برای عملکرد آن حیاتی است. با وجود پیچیدگی درجه دوم، در عمل به دلیل مزایای موازی‌سازی، سریع‌تر از RNN‌های سنتی باقی می‌ماند.

پرسش ۱۱: مدل ترنسفورمر چگونه با مشکل محو شدن گرادیان (vanishing gradient) مقابله می‌کند؟

ترنسفورمرها عمدتاً از طریق استفاده از اتصالات باقی‌مانده (residual/skip connections) و نرمال‌سازی لایه (layer normalization) با مشکل محو شدن گرادیان مقابله می‌کنند. اتصالات باقی‌مانده با افزودن ورودی یک لایه به خروجی آن، به گرادیان‌ها اجازه می‌دهند تا مستقیماً در سراسر شبکه جریان یابند و از کوچک شدن گرادیان‌ها در حین پس‌انتشار (backpropagation) جلوگیری می‌کنند. نرمال‌سازی لایه با نگه داشتن فعال‌سازی‌ها در یک محدوده ثابت، به پایدارسازی آموزش کمک می‌کند، که این امر مقیاس گرادیان‌ها را کنترل می‌کند. علاوه بر این، مکانیزم خود-توجهی امکان جریان مستقیم اطلاعات بین توکن‌ها را فراهم می‌کند، که عمق وابستگی را کاهش داده و مشکل محو شدن گرادیان را بیشتر کاهش می‌دهد.

پرسش ۱۲: توکن‌سازی چیست و چرا در LLM‌ها ضروری است؟

توکن‌سازی فرآیند تبدیل متن خام به واحدهای کوچکتر و گسسته به نام توکن است که می‌تواند کلمات، زیرکلمات یا کاراکترها باشد. این مرحله در LLM‌ها ضروری است زیرا مدل‌ها نمی‌توانند رشته‌های خام متن را مستقیماً پردازش کنند؛ آن‌ها به ورودی عددی نیاز دارند. توکن‌سازی امکان نگاشت متن ورودی به واژگانی از توکن‌های شناخته‌شده را فراهم می‌کند. این توکن‌ها سپس به نمایش‌های عددی به نام جاسازی‌ها (embeddings) تبدیل می‌شوند. این جاسازی‌ها سپس توسط LLM‌ها برای آموزش و استنتاج استفاده می‌شوند و اساساً پردازش متن ورودی را ممکن می‌سازند.

پرسش ۱۳: نقش جاسازی‌های توکن (token embeddings) در مدل ترنسفورمر را توضیح دهید.

جاسازی‌های توکن در مدل ترنسفورمر اطلاعات نحوی و معنایی را کدگذاری می‌کنند. این جاسازی‌ها از یک ماتریس جاسازی به دست می‌آیند، که در آن شاخص هر توکن به یک بردار با اندازه ثابت نگاشت می‌شود. این جاسازی‌های توکن با جاسازی‌های موقعیتی ترکیب می‌شوند تا به مدل در حفظ ترتیب کلمات کمک کنند. این نمایش‌های غنی‌شده سپس به لایه‌های بعدی مدل ترنسفورمر منتقل می‌شوند و امکان درک آگاه از زمینه و تولید متن را فراهم می‌کنند.

پرسش ۱۴: عملکرد لایه جاسازی (embedding layer) در مدل ترنسفورمر را توضیح دهید.

هدف اصلی لایه جاسازی در مدل ترنسفورمر، تبدیل شناسه‌های توکن ورودی به بردارهای متراکم و پیوسته به نام جاسازی است. هر توکن ابتدا به عنوان یک بردار یک-داغ (one-hot vector) نمایش داده شده و سپس با یک ماتریس جاسازی ضرب می‌شود تا جاسازی توکن تولید شود. این جاسازی‌های توکن سپس با جاسازی‌های موقعیتی ترکیب می‌شوند تا اطلاعات مربوط به ترتیب توکن‌ها در توالی حفظ شود. به طور خلاصه، لایه جاسازی توالی توکن‌ها که به عنوان شناسه‌های صحیح نمایش داده شده‌اند را به توالی جاسازی‌ها تبدیل می‌کند. این جاسازی‌ها سپس توسط لایه‌های بعدی پردازش می‌شوند تا اطلاعات متنی به آنها تزریق شود.

پرسش ۱۵: نقش خود-توجهی (self-attention) در مدل ترنسفورمر چیست و چرا به آن «خود-توجهی» می‌گویند؟

مکانیزم خود-توجهی به مدل این امکان را می‌دهد که با اجازه دادن به توکن‌ها برای توجه به تمام توکن‌های موجود در توالی ورودی، یک نمایش غنی از زمینه برای هر توکن محاسبه کند. این نمایش‌های غنی از زمینه، وابستگی‌ها و روابط دوربرد را ثبت می‌کنند. این مکانیزم «خود-توجهی» نامیده می‌شود زیرا توجه در همان توالی محاسبه می‌شود - هر توکن به خود و دیگران بدون ورودی خارجی توجه می‌کند. این فرآیند به ساخت جاسازی‌های متنی غنی کمک می‌کند که برای درک معنا و ساختار در متن حیاتی است.

پرسش ۱۶: هدف رمزگذار (encoder) در یک مدل ترنسفورمر چیست؟

هدف رمزگذار در یک مدل ترنسفورمر پردازش کل توالی ورودی و تولید نمایش‌های عددی آگاه از زمینه است. رمزگذار از خود-توجهی چند-سر (multi-head self-attention) و شبکه‌های عصبی پیش‌خور (feed-forward neural networks) برای ثبت روابط و وابستگی‌ها بین تمام توکن‌ها استفاده می‌کند تا نمایش‌های توکن غنی از زمینه تولید کند. این نمایش‌های توکن غنی از زمینه سپس به رمزگشا (decoder) منتقل می‌شوند که توالی خروجی را تولید می‌کند.

پرسش ۱۷: هدف رمزگشا (decoder) در یک مدل ترنسفورمر چیست؟

هدف رمزگشا در یک مدل ترنسفورمر تولید توالی خروجی بر اساس نمایش‌های متنی ارائه شده توسط رمزگذار و توکن‌های تولید شده قبلی است. لایه‌های رمزگشا از (i) خود-توجهی ماسک‌دار (masked self-attention) برای اطمینان از اینکه پیش‌بینی‌ها برای گام فعلی تنها به خروجی‌های تولید شده قبلی متکی هستند و (ii) توجه رمزگذار-رمزگشا (encoder-decoder attention) برای تمرکز بر روی مرتبط‌ترین بخش‌های خروجی نهایی رمزگذار هنگام تولید هر توکن خروجی استفاده می‌کنند. به عبارت ساده، رمزگشا خروجی رمزگذار (نمایش‌های متنی) را توکن به توکن به خروجی مورد نظر تبدیل می‌کند.

پرسش ۱۸: ساختار رمزگذار-رمزگشا در مدل ترنسفورمر در سطح بالا چگونه کار می‌کند؟

ساختار رمزگذار-رمزگشا در یک ترنسفورمر برای وظایف توالی-به-توالی مانند ترجمه ماشینی استفاده می‌شود. در سطح بالا، رمزگذار کل توالی ورودی را به طور همزمان پردازش می‌کند و با استفاده از خود-توجهی، نمایش‌های برداری غنی و آگاه از زمینه (حالت رمزگذاری شده) ایجاد می‌کند. سپس رمزگشا با استفاده از یک مکانیزم خود-توجهی جداگانه که برای جلوگیری از نگاه به آینده ماسک‌گذاری شده است، توالی خروجی را به صورت خودبازگشتی و توکن به توکن تولید می‌کند. رمزگشا همچنین یک لایه توجه رمزگذار-رمزگشا را در خود جای داده است که به آن اجازه می‌دهد برای هر توکنی که تولید می‌کند، بر روی مرتبط‌ترین بخش‌های خروجی رمزگذار تمرکز کند.

پرسش ۱۹: هدف مقیاس‌بندی (scaling) در مکانیزم خود-توجهی در مدل ترنسفورمر چیست؟

هدف مقیاس‌بندی در مکانیزم خود-توجهی یک مدل ترنسفورمر، پایداری آموزش با کنترل بزرگی مقادیر ضرب داخلی بین بردارهای پرس‌وجو و کلید است. بدون مقیاس‌بندی، ضرب‌های داخلی می‌توانند در فضاها با ابعاد بالا بزرگ شوند و منجر به گرادیان‌های بسیار کوچکی شوند که یادگیری را کند یا ناپایدار می‌کنند. برای جلوگیری از این امر، ضرب‌های داخلی بر جذر ابعاد کلید تقسیم می‌شوند، که این کار امتیازات توجه را در محدوده‌ای نگه می‌دارد که منجر به جریان گرادیان مؤثرتر و بهینه‌سازی پایدارتر می‌شود. بنابراین، مرحله مقیاس‌بندی تضمین می‌کند که مکانیزم خود-توجهی می‌تواند به طور مؤثر بر روی توکن‌های مرتبط توالی ورودی تمرکز کند بدون اینکه در حین آموزش با مشکلات عددی مواجه شود.

پرسش ۲۰: چرا مدل ترنسفورمر به جای یک سر خود-توجهی، از چندین سر خود-توجهی (multiple self-attention heads) استفاده می‌کند؟

مدل‌های ترنسفورمر به جای یک سر خود-توجهی، از چندین سر استفاده می‌کنند زیرا این کار به مدل امکان می‌دهد تا روابط و الگوهای متنوعی را در ورودی ثبت کند. این شبیه به استفاده از چندین چراغ قوه برای روشن کردن جنبه‌های مختلف یک صحنه است. استفاده از یک سر، مدل را به یک شکل از رابطه محدود می‌کند و ظرفیت آن را برای درک کامل داده‌های ورودی کاهش می‌دهد. توجه چند-سر، با اجازه دادن به مدل برای ثبت روابط متنوع، منجر به نمایش‌های متنی غنی‌تر و در نتیجه عملکرد بهتر در سراسر وظایف NLP می‌شود.

پرسش ۲۱: خروجی‌های سرهای چندگانه در توجه چند-سر (multi-head attention) در مدل ترنسفورمر چگونه ترکیب و بازپخش (projected back) می‌شوند؟

در مدل توجه چند-سر ترنسفورمر، خروجی‌های چندین سر خود-توجهی ابتدا به طور مستقل محاسبه می‌شوند و هر کدام یک نمایش وزنی متنی از ورودی را در زیرفضاهای مختلف تولید می‌کنند. این خروجی‌ها سپس در امتداد بُعد ویژگی به هم متصل می‌شوند تا یک بردار ترکیبی واحد تشکیل دهند. این نتیجه متصل شده از طریق یک لایه خطی نهایی (linear projection layer) عبور داده می‌شود که آن را به ابعاد مورد انتظار مدل برای پردازش بیشتر بازپخش می‌کند. این مکانیزم به مدل اجازه می‌دهد تا اطلاعات متنوعی که توسط هر سر ثبت شده است را در یک نمایش یکپارچه ادغام کند.

پرسش ۲۲: خود-توجهی ماسک‌دار (masked self-attention) چه تفاوتی با خود-توجهی معمولی دارد و در کجای ترنسفورمر استفاده می‌شود؟

خود-توجهی ماسک‌دار با محدود کردن مکانیزم توجه، با خود-توجهی معمولی تفاوت دارد، به طوری که هر موقعیت در توالی فقط می‌تواند به موقعیت‌های قبلی و خود توجه کند و از دسترسی به توکن‌های آینده جلوگیری می‌کند. این امر برای وظایف خودبازگشتی (autoregressive) مانند تولید متن، که در آن توکن‌ها به صورت متوالی و یکی‌یکی تولید می‌شوند، ضروری است. خود-توجهی معمولی به هر توکن اجازه می‌دهد به تمام توکن‌های موجود در توالی توجه کند، که در لایه‌های رمزگذار برای درک زمینه به صورت دوطرفه (bidirectionally) مفید است. خود-توجهی ماسک‌دار به طور خاص در لایه‌های رمزگشای مدل ترنسفورمر استفاده می‌شود تا اطمینان حاصل شود که مدل توکن‌ها را یکی‌یکی و بدون نگاه کردن به جلو پیش‌بینی می‌کند.

پرسش ۲۳: مزایا و معایب مکانیزم خود-توجهی در مدل ترنسفورمر را مورد بحث قرار دهید.

خود-توجهی به مدل ترنسفورمر این امکان را می‌دهد که وابستگی‌های دوربرد (روابط متنی) بین توکن‌ها در توالی ورودی را به طور کارآمد ثبت کند. این مکانیزم امکان محاسبات موازی را فراهم می‌کند و سرعت آموزش را در مقایسه با مدل‌های بازگشتی بهبود می‌بخشد. با این حال، پیچیدگی زمانی درجه دوم آن نسبت به طول توالی و ردپای حافظه بزرگ، آن را برای ورودی‌های طولانی از نظر محاسباتی پرهزینه می‌کند.

پرسش ۲۴: هدف از خود-توجهی ماسک‌دار در رمزگشای ترنسفورمر چیست؟

در طول تولید خروجی، خود-توجهی ماسک‌دار در رمزگشا با مسدود کردن دسترسی به توکن‌های آینده، از نگاه کردن مدل به جلو جلوگیری می‌کند. این بدان معناست که هنگام پیش‌بینی یک توکن، مدل محدود به توجه کردن فقط به توکن‌هایی است که قبل از آن در توالی آمده‌اند. این امر (i) رفتار خودبازگشتی را تضمین می‌کند، که به مدل اجازه می‌دهد متن را توکن به توکن تولید کند، و (ii) اطمینان می‌دهد که مدل یاد می‌گیرد پیش‌بینی‌های خود را صرفاً بر اساس زمینه گذشته انجام دهد، و یکپارچگی و صحت مدل‌سازی زبان را حفظ کند.

پرسش ۲۵: توضیح دهید که ماسک‌گذاری در خود-توجهی ماسک‌دار در ترنسفورمر چگونه کار می‌کند.

ماسک‌گذاری در خود-توجهی ماسک‌دار (که در رمزگشای ترنسفورمر استفاده می‌شود) از توجه مدل به توکن‌های آینده در توالی جلوگیری می‌کند. این کار با تنظیم امتیازات توجه برای آن موقعیت‌های آینده به یک عدد منفی بسیار بزرگ (مانند $-\infty$) انجام می‌شود، به طوری که تابع softmax آنها را به صفر تبدیل کند. این امر تضمین می‌کند که پیش‌بینی برای توکن فعلی فقط به توکن‌های شناخته شده قبلی متکی است و خاصیت خودبازگشتی را حفظ می‌کند.

پرسش ۲۶: توضیح دهید چرا به خود-توجهی در رمزگشا، توجه متقاطع (cross-attention) گفته می‌شود. این مکانیزم چه تفاوتی با خود-توجهی در رمزگذار دارد؟

به خود-توجهی در رمزگشا، توجه متقاطع گفته می‌شود زیرا شامل توجه به خروجی رمزگذار هنگام تولید هر توکن در رمزگشا است. به طور خاص، در توجه متقاطع، پرس‌وجوها (queries) از لایه قبلی رمزگشا می‌آیند، در حالی که کلیدها (keys) و مقادیر (values) از خروجی رمزگذار می‌آیند، که به رمزگشا اجازه می‌دهد بر بخش‌های مرتبط توالی ورودی تمرکز کند. این با خود-توجهی در رمزگذار متفاوت است، جایی که مکانیزم توجه صرفاً در داخل خود توالی ورودی عمل می‌کند و پرس‌وجوها، کلیدها و مقادیر همگی از داده‌های یکسانی مشتق می‌شوند. بنابراین، خود-توجهی رمزگذار وابستگی‌های داخلی را ثبت می‌کند، در حالی که توجه متقاطع رمزگشا، رمزگشا را برای تولید متنی به نمایش‌های رمزگذار پیوند می‌دهد.

پرسش ۲۷: تابع softmax چیست و در کجای مدل ترنسفورمر اعمال می‌شود؟

تابع softmax یک عملیات ریاضی است که یک بردار از امتیازات خام (logits) را به یک توزیع احتمال تبدیل می‌کند، به طوری که هر مقدار بین ۰ و ۱ قرار می‌گیرد و مجموع تمام مقادیر برابر با ۱ است. در مدل ترنسفورمر، softmax در لایه خروجی هنگام پیش‌بینی توکن اعمال می‌شود تا خروجی‌های امتیاز خام مدل را به احتمالات روی واژگان تبدیل کند و انتخاب محتمل‌ترین توکن بعدی را ممکن سازد. علاوه بر این، softmax در مکانیزم خود-توجهی برای نرمال‌سازی وزن‌های توجه استفاده می‌شود. این نرمال‌سازی به مدل اجازه می‌دهد تا اهمیت توکن‌های مختلف ورودی را به طور مؤثر وزن‌دهی کند. بنابراین، softmax نقشی حیاتی در مدیریت احتمالات هم در خود-توجهی و هم در پیش‌بینی‌های نهایی ایفا می‌کند.

پرسش ۲۸: هدف از اتصالات باقی‌مانده (residual/skip connections) در لایه‌های مدل ترنسفورمر چیست؟

هدف اصلی اتصالات باقی‌مانده (skip) در لایه‌های ترنسفورمر این است که به گرادین‌ها اجازه دهند در حین پس‌انتشار (backpropagation) مستقیماً در شبکه جریان یابند. این امر به کاهش مشکل محو شدن گرادین در شبکه‌های عمیق کمک می‌کند و آموزش مؤثر را تضمین می‌کند. به طور خاص، آنها این امکان را فراهم می‌کنند که خروجی یک لایه به جای اینکه فقط خروجی لایه باشد، برابر با "ورودی لایه + خروجی لایه" باشد. این بدان معناست که لایه به جای یک نمایش کاملاً جدید، یک تغییر افزایشی را یاد می‌گیرد. این امر آموزش معماری‌های بسیار عمیق را تسهیل می‌کند و منجر به عملکرد کلی بهتر می‌شود.

پرسش ۲۹: چرا از نرمال‌سازی لایه (layer normalization) استفاده می‌شود و در کجای ترنسفورمر اعمال می‌شود؟

نرمال‌سازی لایه در ترنسفورمر برای پایداری و تسریع آموزش با نرمال‌سازی ورودی‌ها در سراسر ویژگی‌ها در هر لایه استفاده می‌شود و اطمینان می‌دهد که فعال‌سازی‌ها دارای میانگین صفر و واریانس یک هستند. این نرمال‌سازی به کنترل مقیاس گرادینت‌ها در حین پس‌انتشار کمک می‌کند و از مسائلی مانند انفجار یا محو شدن گرادینت‌ها جلوگیری می‌کند. در معماری‌های ترنسفورمر، نرمال‌سازی لایه قبل از زیرلایه‌های خود-توجهی و پیش‌خور (Pre-LN) یا بعد از این زیرلایه‌ها در داخل اتصالات باقی‌مانده (Post-LN) اعمال می‌شود. نوع Pre-LN پایداری گرادینت را بهبود می‌بخشد و آموزش سریع‌تر را بدون نیاز به گرم کردن نرخ یادگیری (learning rate warm-up) ممکن می‌سازد، در حالی که Post-LN در ترنسفورمر اصلی استفاده شد اما در صورت عدم مدیریت دقیق می‌تواند منجر به گرادینت‌های ناپایدار شود.

پرسش ۳۰: خطای آنتروپی متقاطع (cross-entropy loss) چیست و چگونه در طول آموزش ترنسفورمر اعمال می‌شود؟

خطای آنتروپی متقاطع تفاوت بین توزیع احتمال پیش‌بینی شده یک مدل و توزیع واقعی برچسب‌های هدف را اندازه‌گیری می‌کند. در آموزش ترنسفورمر، این خطا با مقایسه لاجیت‌های خروجی مدل (پس از softmax) با شاخص توکن واقعی، میزان خوب بودن پیش‌بینی مدل برای توکن بعدی را کمی‌سازی می‌کند. این خطا پیش‌بینی‌های نادرست را با مقادیر بالاتر جریمه می‌کند و مدل را تشویق می‌کند تا احتمال بیشتری به توکن صحیح اختصاص دهد. به حداقل رساندن خطای آنتروپی متقاطع از طریق پس‌انتشار به بهینه‌سازی پارامترهای مدل برای تولید توالی دقیق کمک می‌کند.

پرسش ۳۱: ترنسفورمرها و RNN‌ها را از نظر مدیریت وابستگی‌های دوربرد (long-range dependencies) مقایسه کنید.

ترنسفورمرها به دلیل تفاوت اساسی در معماری، در ثبت وابستگی‌های دوربرد اساساً بهتر از شبکه‌های عصبی بازگشتی (RNN) هستند. RNN‌ها توالی‌ها را به صورت متوالی پردازش می‌کنند و از مشکل محو شدن گرادینت رنج می‌برند، که باعث می‌شود اطلاعات از گام‌های اولیه در توالی‌های طولانی کاهش یابد و ارتباط دادن توکن‌های دور از هم دشوار شود. در مقابل، ترنسفورمرها از یک مکانیزم خود-توجهی استفاده می‌کنند که یک امتیاز رابطه مستقیم بین هر جفت توکن در توالی محاسبه می‌کند. این به آن اجازه می‌دهد تا وابستگی‌ها را بدون توجه به فاصله بین توکن‌ها در $O(1)$ گام متوالی ثبت کند، برخلاف $O(n)$ گام مورد نیاز RNN‌ها. این توجه غیرمتوالی و موازی به ترنسفورمرها اجازه می‌دهد تا زمینه را در بازه‌های بسیار طولانی‌تر به طور مؤثر مدل‌سازی کنند.

پرسش ۳۲: محدودیت‌های اساسی مدل ترنسفورمر چیست؟

محدودیت‌های اساسی مدل ترنسفورمر شامل نیازهای بالای محاسباتی و حافظه است. این به دلیل پیچیدگی درجه دوم مکانیزم خود-توجهی است که پردازش توالی‌های بسیار طولانی را چالش‌برانگیز می‌کند. علاوه بر این، آنها به مقادیر عظیمی از داده‌های آموزشی متنوع نیاز دارند و می‌توانند به داده‌های مغرضانه یا با کیفیت پایین حساس باشند، که بر تعمیم‌پذیری و استحکام آنها تأثیر می‌گذارد. با وجود این چالش‌ها، آنها همچنان قدرتمند و بهتر از مدل‌های یادگیری عمیق سنتی مانند CNN و RNN‌ها هستند.

پرسش ۳۳: ترنسفورمرها چگونه محدودیت‌های مدل‌های یادگیری عمیق سنتی مانند CNN و RNN‌ها را برطرف می‌کنند؟

ترنسفورمرها محدودیت‌های مدل‌های یادگیری عمیق سنتی مانند CNN و RNN‌ها را عمدتاً از طریق مکانیزم خود-توجهی و قابلیت پردازش موازی خود برطرف می‌کنند. RNN‌ها داده‌ها را به صورت متوالی پردازش می‌کنند و به دلیل محو شدن گرادیان‌ها با وابستگی‌های دوربرد مشکل دارند. CNN‌ها در استخراج ویژگی‌های محلی عالی هستند اما فاقد توانایی مدل‌سازی روابط متوالی یا دوربرد در سراسر داده‌ها هستند. مکانیزم خود-توجهی به ترنسفورمرها اجازه می‌دهد تا تمام بخش‌های ورودی را به صورت موازی پردازش کنند و اهمیت هر کلمه/توکن را برای هر کلمه/توکن دیگر بدون توجه به فاصله آنها ارزیابی کنند. در مقایسه با مدل‌های سنتی، ترنسفورمرها می‌توانند بهتر مقیاس‌پذیر باشند و بر روی مجموعه داده‌های بزرگ سریع‌تر آموزش ببینند.

پرسش ۳۴: مدل ترنسفورمر چگونه با مشکل محو شدن گرادیان (vanishing gradient) مقابله می‌کند؟

ترنسفورمرها عمدتاً از طریق استفاده از اتصالات باقی‌مانده (residual/skip connections) و نرمال‌سازی لایه (layer normalization) با مشکل محو شدن گرادیان مقابله می‌کنند. اتصالات باقی‌مانده با افزودن ورودی یک لایه به خروجی آن، به گرادیان‌ها اجازه می‌دهند تا مستقیماً در سراسر شبکه جریان یابند و از کوچک شدن گرادیان‌ها در حین پس‌انتشار (backpropagation) جلوگیری می‌کنند. نرمال‌سازی لایه با نگه داشتن فعال‌سازی‌ها در یک محدوده ثابت، به پایدارسازی آموزش کمک می‌کند، که این امر مقیاس گرادیان‌ها را کنترل می‌کند. علاوه بر این، مکانیزم خود-توجهی امکان جریان مستقیم اطلاعات بین توکن‌ها را فراهم می‌کند، که عمق وابستگی را کاهش داده و مشکل محو شدن گرادیان را بیشتر کاهش می‌دهد.

پرسش ۳۵: هدف زیرلایه پیش‌خور موقعیت-محور (position-wise feed-forward sublayer) در مدل ترنسفورمر چیست؟

زیرلایه پیش‌خور موقعیت-محور در مدل ترنسفورمر برای پردازش مستقل نمایش هر موقعیت در توالی پس از مکانیزم توجه عمل می‌کند. این زیرلایه شامل دو تبدیل خطی با یک فعال‌سازی ReLU در بین آنها است که به طور یکسان و جداگانه به هر بردار توکن اعمال می‌شود. این زیرلایه غیرخطی بودن را اضافه کرده و ظرفیت مدل را برای یادگیری ویژگی‌های پیچیده با تبدیل خروجی توجه به نمایش‌های غنی‌تر برای لایه‌های بعدی افزایش می‌دهد.

پرسش ۳۶: تفاوت بین آموزش LLM و استنتاج LLM را به طور خلاصه توضیح دهید.

آموزش LLM فرآیندی است که در آن یک مدل زبان بزرگ الگوها و روابط را از مجموعه داده‌های عظیم یاد می‌گیرد و پارامترهای داخلی (وزن‌ها) خود را برای به حداقل رساندن خطای پیش‌بینی تنظیم می‌کند. استنتاج LLM فاز استقرار است که در آن مدل آموزش‌دیده با پارامترهای ثابت از دانش آموخته شده خود برای تولید پاسخ برای یک ورودی جدید و دیده‌نشده استفاده می‌کند. آموزش از نظر محاسباتی فشرده است و یک بار انجام می‌شود، در حالی که استنتاج سریع است و هر بار که از مدل استفاده می‌شود، اتفاق می‌افتد.

پرسش ۳۷: تأخیر (latency) در استنتاج LLM چیست و چرا مهم است؟

تأخیر در استنتاج LLM به تأخیر زمانی بین ارسال یک ورودی به مدل و دریافت خروجی نهایی و کامل اشاره دارد. این معیار سرعت پردازش ورودی و تولید خروجی توسط مدل را اندازه‌گیری می‌کند و معمولاً بر حسب میلی‌ثانیه یا ثانیه بیان می‌شود. این یک معیار عملکرد حیاتی است زیرا مستقیماً بر تجربه کاربر و توان عملیاتی برنامه‌ها تأثیر می‌گذارد. تأخیر کم برای ارائه تجربیات روان و بی‌درنگ در چت‌بات‌ها، دستیاران کدنویسی یا سیستم‌های جستجو بسیار مهم است. تأخیر بالا می‌تواند منجر به تعاملات کند و خسته‌کننده شود، به خصوص در سیستم‌های بی‌درنگ یا مکالمه‌ای.

پرسش ۳۸: استنتاج دسته‌ای (batch inference) چیست و چه تفاوتی با استنتاج تک-پرس‌وجویی (single-query inference) دارد؟

در استنتاج دسته‌ای، LLMها خروجی‌ها را برای مجموعه بزرگی از داده‌های ورودی انباشته شده (دسته) به یکباره پردازش و تولید می‌کنند. استنتاج دسته‌ای برای کارهای آفلاین یا ناهمزمان مانند طبقه‌بندی دسته‌ای اسناد یا تولید توصیه‌ها در طول شب بسیار کارآمد است. این در تضاد با استنتاج تک-پرس‌وجویی (یا استنتاج بی‌درنگ) است، که در آن مدل نقاط داده فردی را به محض رسیدن پردازش می‌کند و پاسخ‌ها را با تأخیر بسیار کم ارائه می‌دهد. استنتاج تک-پرس‌وجویی معمولاً برای برنامه‌های تعاملی و رو به کاربر مانند چت‌بات‌ها مورد نیاز است. تفاوت اصلی در الزامات تأخیر و بهینه‌سازی توان عملیاتی نهفته است: استنتاج دسته‌ای توان عملیاتی بالا (پردازش حجم زیادی از داده‌ها به سرعت) را در اولویت قرار می‌دهد در حالی که تأخیر کم را فدا می‌کند، در حالی که استنتاج تک-پرس‌وجویی تأخیر کمینه را در اولویت قرار می‌دهد.

پرسش ۳۹: دسته‌بندی (batching) به طور کلی چگونه به کارایی استنتاج LLM کمک می‌کند؟

دسته‌بندی با فعال کردن پردازش موازی چندین درخواست، کارایی استنتاج LLM را به طور قابل توجهی بهبود می‌بخشد، که این امر بهره‌وری GPU را به حداکثر می‌رساند. به جای پردازش هر پرس‌وجو به صورت متوالی، دسته‌بندی درخواست‌ها را با هم گروه‌بندی می‌کند تا از ظرفیت محاسباتی کامل سخت‌افزار استفاده کند و منجر به توان عملیاتی بالاتر (توکن در ثانیه) شود. دسته‌بندی پیوسته (Continuous batching) با استفاده از زمان‌بندی در سطح تکرار، کارایی را بیشتر افزایش می‌دهد، جایی که درخواست‌های جدید می‌توانند جایگزین درخواست‌های تکمیل شده در یک دسته شوند بدون اینکه منتظر پایان یافتن تمام توالی‌ها باشند، که این امر به بهبودهای بسیار بالاتر در توان عملیاتی منجر می‌شود. این رویکرد بهره‌وری GPU را از پردازش متوالی با بهره‌وری کم به اجرای موازی بهینه تبدیل می‌کند و آن را برای سرویس‌دهی LLM در محیط تولید ضروری می‌سازد.

پرسش ۴۰: بده‌بستان (trade-offs) بین دسته‌بندی و تأخیر در سرویس‌دهی LLM را توضیح دهید.

دسته‌بندی در سرویس‌دهی LLM با پردازش همزمان چندین درخواست، توان عملیاتی را افزایش می‌دهد و بهره‌وری GPU را به حداکثر می‌رساند، به خصوص در بارهای کاری سنگین. با این حال، دسته‌های بزرگتر به ناچار تأخیر را افزایش می‌دهند زیرا درخواست‌های سریع‌تر یا کوتاه‌تر باید منتظر بمانند تا طولانی‌ترین توالی به پایان برسد تا دسته تمام شود. استراتژی‌های دسته‌بندی پیوسته یا پویا (continuous or dynamic batching) سعی می‌کنند با تنظیم دسته‌ها در زمان واقعی، این تعادل را برقرار کنند و درخواست‌های تکمیل شده را با درخواست‌های جدید جایگزین کنند تا کارایی را حفظ کرده و زمان انتظار را به حداقل برسانند. در نهایت، این بده‌بستان به نیازهای برنامه بستگی دارد—سیستم‌های تعاملی تأخیر کم را در اولویت قرار می‌دهند، در حالی که کارهای صنعتی یا آفلاین به دنبال توان عملیاتی بالاتر با هزینه مقداری تأخیر هستند.

پرسش ۴۱: چگونه تکنیک‌هایی مانند ترکیب متخصصان (mixture-of-experts - MoE) می‌توانند کارایی استنتاج را بهینه کنند؟

ترکیب متخصصان (MoE) کارایی استنتاج را با معرفی محاسبات شرطی یا پراکندگی (sparsity) بهینه می‌کند. به جای فعال کردن تمام پارامترهای مدل برای هر توکن ورودی، یک شبکه دروازه‌بندی (gating network) سبک به صورت پویا فقط زیرمجموعه کوچکی از زیرشبکه‌های تخصصی "متخصص" را برای پردازش توکن انتخاب می‌کند. این فعال‌سازی انتخابی به این معنی است که مدل به طور قابل توجهی عملیات ممیز شناور (FLOPs) کمتری در هر توکن در مقایسه با یک مدل متراکم با اندازه کل مشابه انجام می‌دهد. این امر منجر به تولید سریع‌تر توکن (کاهش تأخیر) و توان عملیاتی بالاتر می‌شود.

پرسش ۴۲: نقش استراتژی رمزگشایی (decoding strategy) در تولید متن LLM را توضیح دهید.

استراتژی‌های رمزگشایی در LLMها، قواعد تصمیمی هستند که یک مدل برای انتخاب توکن بعدی از واژگان خود هنگام تولید متن استفاده می‌کند. یک LLM فقط کلمه بعدی را "نمی‌داند"؛ بلکه برای هر کلمه ممکن که می‌شناسد یک احتمال محاسبه می‌کند. استراتژی رمزگشایی صرفاً روشی است که برای انتخاب کلمه نهایی از آن لیست احتمالات برای تشکیل متن خروجی، کلمه به کلمه، استفاده می‌شود. انتخاب استراتژی به شدت بر این تأثیر می‌گذارد که خروجی قابل پیش‌بینی و واقعی باشد یا خلاقانه و متنوع. استراتژی‌های رایج رمزگشایی شامل جستجوی حریصانه (greedy search)، جستجوی پرتویی (beam search) و روش‌های نمونه‌برداری مانند مقیاس‌بندی دما و نمونه‌برداری top-k/top-p است.

پرسش ۴۳: استراتژی‌های مختلف رمزگشایی در LLM‌ها کدامند؟

استراتژی‌های رمزگشایی در LLM‌ها نحوه انتخاب توکن‌ها در طول تولید متن را تعیین می‌کنند. استراتژی‌های اصلی عبارتند از:

- **رمزگشایی حریصانه (Greedy Decoding):** همیشه توکنی را انتخاب می‌کند که در هر مرحله بالاترین احتمال را دارد. سریع است اما می‌تواند متن تکراری یا غیربهبوده تولید کند.
- **جستجوی پرتویی (Beam Search):** چندین توالی کاندیدای برتر ("پرتو") را در هر مرحله نگه می‌دارد و محتمل‌ترین توالی کلی را انتخاب می‌کند. کیفیت را بهبود می‌بخشد اما محاسبات را افزایش می‌دهد.
- **نمونه‌برداری Top-k:** توکن بعدی را به صورت تصادفی از بین k توکن محتمل برتر نمونه‌برداری می‌کند، که تنوع را افزایش داده و تکرار را کاهش می‌دهد.
- **نمونه‌برداری Top-p (Nucleus):** به صورت تصادفی از کوچکترین مجموعه توکن‌هایی که احتمال تجمعی آنها $p \leq$ (مثلاً ۰.۹) است، نمونه‌برداری می‌کند.
- **نمونه‌برداری دما (Temperature Sampling):** تصادفی بودن را با مقیاس‌بندی لاجیت‌ها قبل از softmax تنظیم می‌کند - دمای بالاتر ($1 <$) خروجی‌ها را تصادفی‌تر می‌کند؛ دمای پایین‌تر ($1 >$) آنها را قطعی‌تر می‌کند.
- **رمزگشایی گمانه‌زانه (Speculative Decoding):** از یک مدل پیش‌نویس کوچکتر برای پیش‌بینی چندین توکن در آینده استفاده می‌کند و به سرعت آنها را با مدل اصلی تأیید می‌کند، که تولید را بدون از دست دادن کیفیت به شدت تسریع می‌کند.

پرسش ۴۴: تأثیر استراتژی رمزگشایی بر کیفیت و تأخیر خروجی تولید شده توسط LLM را توضیح دهید.

استراتژی‌های رمزگشایی با ایجاد تعادل بین کیفیت (انسجام، تنوع و ارتباط) و تأخیر (سرعت تولید)، تأثیر حیاتی بر خروجی LLM دارند. روش‌های قطعی مانند جستجوی حریصانه به دلیل انتخاب توکن با بالاترین احتمال در هر مرحله سریع هستند اما اغلب متن تکراری و با کیفیت پایین تولید می‌کنند. جستجوی پرتویی با کاوش چندین توالی توکن کیفیت را بهبود می‌بخشد اما به دلیل مدیریت چندین پرتو، تأخیر را افزایش می‌دهد. روش‌های رمزگشایی تصادفی، مانند نمونه‌برداری Top-k یا Top-p (nucleus)، متن را توکن به توکن تولید می‌کنند و هر توکن بعدی را به صورت احتمالی و نه قطعی انتخاب می‌کنند. در حالی که این کار خلاقیت و تنوع را افزایش می‌دهد، سربار محاسباتی اضافی (مرتب‌سازی یا جمع تجمعی) را در هر مرحله رمزگشایی به همراه دارد که تأخیر را افزایش می‌دهد. تکنیک‌های جدیدتر و سریع‌تر مانند رمزگشایی گمانه‌زانه با استفاده از یک مدل کوچکتر برای پیش‌نویس توکن‌ها، تأخیر را کاهش می‌دهند، که مدل بزرگتر آنها را به صورت موازی تأیید می‌کند و سرعت قابل توجهی را ارائه می‌دهد در حالی که هدف آن حفظ کیفیت بالای خروجی مدل اصلی است.

پرسش ۴۵: استراتژی رمزگشایی جستجوی حریصانه (greedy search) و عیب اصلی آن را توضیح دهید.

استراتژی رمزگشایی جستجوی حریصانه در LLMها یک روش ساده است که در آن مدل در هر مرحله از فرآیند تولید، توکنی را که بالاترین احتمال را دارد به عنوان کلمه بعدی انتخاب می‌کند. این رویکرد بسیار کارآمد و قطعی است و همیشه برای یک ورودی معین، خروجی یکسانی تولید می‌کند. با این حال، عیب اصلی آن کوتاه‌بینی آن است. با تمرکز تنها بر روی انتخاب بهینه محلی، اغلب در یافتن توالی‌ای که بالاترین احتمال کلی را دارد شکست می‌خورد و به طور بالقوه منجر به متن تکراری، کمتر منسجم یا غیربهره‌مند می‌شود.

پرسش ۴۶: جستجوی پرتویی (Beam Search) چگونه جستجوی حریصانه (Greedy Search) را بهبود می‌بخشد و نقش پارامتر عرض پرتو (beam width) چیست؟

جستجوی پرتویی با کاوش مجموعه وسیع‌تری از احتمالات، جستجوی حریصانه را بهبود می‌بخشد. این روش به جای اینکه مانند جستجوی حریصانه فقط محتمل‌ترین توالی را نگه دارد، k توالی جزئی محتمل‌تر را (که در آن k عرض پرتو است) در هر مرحله رمزگشایی ردیابی می‌کند. این کار با کاهش خطر گیر افتادن در انتخاب‌های اولیه که به صورت محلی بهینه اما در نهایت غیربهره‌مند هستند، شانس یافتن یک توالی بهتر در سطح جهانی را به طور قابل توجهی افزایش می‌دهد. پارامتر عرض پرتو (k) تعداد فرضیه‌های نگهداری شده را تعیین می‌کند؛ عرض پرتو بزرگتر گزینه‌های بیشتری را کاوش می‌کند و به طور کلی نتایج بهتری را با هزینه پیچیدگی محاسباتی بالاتر و سرعت رمزگشایی کندتر به همراه دارد.

پرسش ۴۷: چه زمانی یک استراتژی قطعی (مانند جستجوی پرتویی) به یک استراتژی تصادفی (نمونه‌برداری) ترجیح داده می‌شود؟ یک مورد استفاده خاص ارائه دهید.

یک استراتژی قطعی مانند جستجوی پرتویی زمانی ترجیح داده می‌شود که ثبات، قابلیت اطمینان و تکرارپذیری خروجی‌ها حیاتی باشد. این روش به طور سیستماتیک محتمل‌ترین توالی‌های توکن را کاوش می‌کند و آن را برای وظایفی مانند ترجمه ماشینی، خلاصه‌سازی یا تولید کد که در آنها دقت و انسجام بر تنوع ارجحیت دارد، ایده‌آل می‌سازد. به عنوان مثال، در خلاصه‌سازی متون حقوقی یا پزشکی، جستجوی پرتویی ثبات واقعی را تضمین می‌کند و از تغییرات تصادفی که روش‌های مبتنی بر نمونه‌برداری ممکن است ایجاد کنند، جلوگیری می‌کند.

پرسش ۴۸: بده‌بستان اصلی بین هزینه محاسباتی و کیفیت خروجی را هنگام مقایسه جستجوی حریصانه و جستجوی پرتویی مورد بحث قرار دهید.

جستجوی حریصانه از نظر محاسباتی ارزان است زیرا در هر مرحله فقط بهترین توکن را در نظر می‌گیرد و منجر به زمان رمزگشایی سریع با حداقل استفاده از حافظه می‌شود. با این حال، اغلب منجر به یک توالی خروجی غیربهره‌مند می‌شود زیرا نمی‌تواند از انتخاب‌هایی که به صورت محلی خوب اما در سطح جهانی ضعیف هستند، بهبود یابد. در مقابل، جستجوی پرتویی k (عرض پرتو) توالی جزئی امیدوارکننده‌تر را در هر مرحله حفظ می‌کند که هزینه محاسباتی و زمان رمزگشایی و استفاده از حافظه را به طور قابل توجهی افزایش می‌دهد. با این حال، این کاوش گسترده‌تر در فضای جستجو، احتمال یافتن یک توالی با کیفیت بالاتر و بهینه‌تر در سطح جهانی را به طرز چشمگیری بهبود می‌بخشد.

پرسش ۴۹: وقتی دما (temperature) را روی ۰.۰ تنظیم می‌کنید، از کدام استراتژی رمزگشایی استفاده می‌کنید؟

تنظیم دما روی ۰.۰ به این معنی است که شما از رمزگشایی حریصانه (greedy decoding) استفاده می‌کنید. این استراتژی مدل را مجبور می‌کند که همیشه توکنی را انتخاب کند که در هر مرحله بالاترین احتمال مطلق را دارد و خروجی را قطعی می‌کند. این کار اساساً تمام تصادفی بودن را حذف می‌کند و به محتمل‌ترین توالی بر اساس توزیع آموخته شده مدل اولویت می‌دهد.

پرسش ۵۰: جستجوی پرتویی (Beam Search) اساساً چه تفاوتی با جستجوی اول-سطح (BFS) یا جستجوی اول-عمق (DFS) دارد؟

جستجوی پرتویی با جستجوی اول-سطح (BFS) و جستجوی اول-عمق (DFS) متفاوت است زیرا یک الگوریتم جستجوی جامع نیست، بلکه یک جستجوی تقریبی مبتنی بر هیوریستیک است. برخلاف BFS که تمام گره‌ها را سطح به سطح کاوش می‌کند، یا DFS که یک مسیر را تا حد امکان عمیق کاوش می‌کند، جستجوی پرتویی در هر مرحله فقط k کاندیدای برتر و امیدوارکننده‌تر را نگه می‌دارد. این کاوش حریصانه و محدود، آن را از نظر حافظه و زمان به طور قابل توجهی کارآمدتر از BFS یا DFS جامع می‌کند، که برای فضاهای حالت بزرگ حیاتی است. با این حال، کامل بودن و بهینگی را فدا می‌کند، زیرا ممکن است بهترین راه حل هرس شود.

پرسش ۵۱: معیارهای انتخاب استراتژی‌های مختلف رمزگشایی را توضیح دهید.

معیارهای انتخاب استراتژی‌های مختلف رمزگشایی، مانند جستجوی حریصانه، جستجوی پرتویی، یا روش‌های نمونه‌برداری (مانند nucleus یا top-k sampling)، عمدتاً به تعادل مورد نظر بین کیفیت خروجی، تنوع و هزینه محاسباتی بستگی دارد. جستجوی حریصانه سریع است اما اغلب متن غیربهبوده، تکراری یا بی‌روح تولید می‌کند، که آن را برای مواقعی که سرعت حیاتی است و کیفیت بالا در اولویت نیست، مناسب می‌سازد. جستجوی پرتویی با کاوش چندین مسیر کیفیت را بهبود می‌بخشد و برای وظایفی که به خروجی‌های قطعی نیاز دارند، مانند ترجمه یا خلاصه‌سازی، ترجیح داده می‌شود، هرچند کندتر است. روش‌های نمونه‌برداری زمانی انتخاب می‌شوند که هدف تولید خروجی‌های خلاقانه‌تر، متنوع‌تر و کمتر قطعی باشد، مانند سیستم‌های گفتگو یا داستان‌سرایی، با معرفی تصادفی بودن کنترل‌شده.

پرسش ۵۲: روش‌های رمزگشایی قطعی و تصادفی را در LLM‌ها مقایسه کنید.

روش‌های رمزگشایی قطعی، مانند جستجوی حریصانه و جستجوی پرتویی، به طور مداوم برای ورودی یکسان خروجی یکسانی تولید می‌کنند زیرا توکن بعدی را صرفاً بر اساس بالاترین احتمال یا مجموعه‌ای ثابت از مسیرهای با احتمال بالا و بدون هیچ عنصر تصادفی انتخاب می‌کنند. در مقابل، روش‌های رمزگشایی تصادفی، مانند نمونه‌برداری با دما، نمونه‌برداری Top-K و نمونه‌برداری (Nucleus) Top-p، تصادفی بودن را به فرآیند انتخاب توکن اضافه می‌کنند و منجر به خروجی‌های متنوع و متفاوت حتی برای ورودی‌های یکسان می‌شوند. روش‌های قطعی به طور کلی برای وظایفی که نیاز به پیش‌بینی‌پذیری و دقت واقعی دارند (مانند ترجمه، خلاصه‌سازی) ترجیح داده می‌شوند، در حالی که روش‌های تصادفی در وظایف باز یا خلاقانه (مانند داستان‌سرایی، طوفان فکری) که در آنها تازگی مطلوب است، برتری دارند.

پرسش ۵۳: نقش پنجره زمینه (context window) در حین استنتاج LLM چیست؟

پنجره زمینه به حداکثر تعداد کل توکن‌ها (هم ورودی و هم خروجی) اشاره دارد که مدل می‌تواند در یک زمان پردازش کند. پنجره زمینه در حین استنتاج LLM به عنوان حافظه کاری مدل عمل می‌کند. این پنجره به مدل امکان می‌دهد تا هم ورودی فعلی و هم توکن‌های قبلی را برای تولید پاسخ‌های مرتبط و منسجم از نظر متنی در نظر بگیرد. یک پنجره زمینه بزرگ‌تر به LLM اجازه می‌دهد تا با حفظ اطلاعات بیشتر، متون طولانی‌تر یا وظایف پیچیده‌تر را مدیریت کند. با این حال، اگر ورودی از اندازه پنجره فراتر رود، مدل توکن‌های قبلی را "فراموش" می‌کند و به طور بالقوه دقت پاسخ را کاهش می‌دهد. اساساً، این پنجره تعیین می‌کند که مدل چه مقدار اطلاعات گذشته را می‌تواند برای پیش‌بینی بعدی استفاده کند.

پرسش ۵۴: مزایا و معایب پنجره‌های زمینه بزرگ و کوچک را در استنتاج LLM توضیح دهید.

پنجره زمینه در LLM‌ها حداکثر تعداد توکن‌هایی را که می‌تواند در یک زمان پردازش کند، تعریف می‌کند و مانند حافظه کاری مدل است. یک پنجره زمینه بزرگ به مدل اجازه می‌دهد تا ورودی بیشتری را پردازش کند و منجر به خروجی‌های منسجم‌تر و آگاه‌تر از زمینه شود. با این حال، پردازش توکن‌های ورودی بیشتر به طور قابل توجهی مصرف حافظه، هزینه محاسباتی و تأخیر را افزایش می‌دهد. در مقابل، یک پنجره زمینه کوچک سریع‌تر و از نظر حافظه کارآمدتر است و آن را برای محیط‌های با منابع کمتر یا برنامه‌های بی‌درنگ مناسب می‌سازد. با این حال، این پنجره "حافظه" مدل را محدود می‌کند و به طور بالقوه باعث می‌شود که اطلاعات قبلی را از دست بدهد و خروجی‌های کمتر سازگار یا مرتبط تولید کند.

پرسش ۵۵: هدف دما (temperature) در استنتاج LLM چیست و چگونه بر خروجی تأثیر می‌گذارد؟

پارامتر دما در استنتاج LLM با تغییر مقیاس لاجیت‌ها قبل از تابع softmax، تصادفی بودن فرآیند انتخاب توکن را کنترل می‌کند. هدف آن مدیریت بده‌بستان بین خلاقیت و قطعیت در خروجی تولید شده است. دمای بالا (مثلاً نزدیک به ۱.۰) توزیع احتمال را مسطح‌تر می‌کند و شانس انتخاب توکن‌های کمتر محتمل را افزایش می‌دهد، که منجر به متن متنوع‌تر، خلاقانه‌تر و گاهی کمتر منسجم می‌شود. در مقابل، دمای پایین (مثلاً نزدیک به ۰.۰) مدل را قطعی‌تر و متمرکز بر محتمل‌ترین توکن‌ها می‌کند و متن منسجم‌تری تولید می‌کند.

پرسش ۵۶: تولید خودبازگشتی (autoregressive generation) در زمینه LLM‌ها چیست؟

تولید خودبازگشتی فرآیند متوالی است که توسط آن LLM‌ها متن را، توکن به توکن (کلمه یا واحد زیرکلمه)، با استفاده از توکن‌های تولید شده قبلی و ورودی اصلی به عنوان زمینه تولید می‌کنند. اساساً، مدل محتمل‌ترین توکن بعدی را بر اساس تمام توکن‌های قبلی پیش‌بینی می‌کند و یک زنجیره وابستگی ایجاد می‌کند که منجر به خروجی متنی منسجم و مرتبط با زمینه می‌شود. این فرآیند تا زمانی ادامه می‌یابد که یک توکن پایان توالی پیش‌بینی شود یا به یک محدودیت طول از پیش تعریف شده برسد.

پرسش ۵۷: نقاط قوت و محدودیت‌های تولید متن خودبازگشتی در LLMها را توضیح دهید.

نقطه قوت اصلی تولید متن خودبازگشتی در LLMها توانایی آن در تولید متن متوالی منسجم، مرتبط با زمینه و با کیفیت بالا است. این به دلیل روش آن در پیش‌بینی توکن بعدی بر اساس تمام توکن‌های قبلی اتفاق می‌افتد. با این حال، محدودیت اصلی آن این است که ذاتاً کند است زیرا هر توکن جدید باید به صورت سریالی تولید شود، که مانع از موازی‌سازی واقعی می‌شود. علاوه بر این، در فرآیند تولید متوالی، خطاها می‌توانند با گذشت زمان انباشته شوند زیرا هر پیش‌بینی به توکن‌های قبلی وابسته است.

پرسش ۵۸: توضیح دهید که مدل‌های زبان انتشاری (Diffusion Language Models - DLMS) چه تفاوتی با مدل‌های زبان بزرگ (LLMs) دارند.

مدل‌های زبان انتشاری (DLMS) در نحوه تولید متن با مدل‌های زبان بزرگ سنتی (LLMs) متفاوت هستند. LLMها از تولید خودبازگشتی استفاده می‌کنند و یک توکن را در هر زمان بر اساس توکن‌های قبلی پیش‌بینی می‌کنند. DLMها از یک فرآیند نویززدایی استفاده می‌کنند که از نویز تصادفی شروع شده و به طور مکرر آن را به متن منسجم اصلاح می‌کند—شبيه به نحوه تولید تصاویر توسط مدل‌های انتشاری. این رویکرد به DLMها اجازه می‌دهد تا زمینه کلی را به طور مؤثرتری درک کنند و به طور بالقوه خروجی‌های متنوع‌تری تولید کنند.

پرسش ۵۹: آیا برای برنامه‌های حساس به تأخیر (latency-sensitive) DLMها را ترجیح می‌دهید یا LLMها را؟

LLMها خودبازگشتی هستند و متن را به صورت متوالی و توکن به توکن تولید می‌کنند، که یک گلوگاه متوالی ایجاد می‌کند و منجر به تأخیر کلی کندتر برای خروجی‌های طولانی می‌شود. در مقابل، DLMها غیرخودبازگشتی هستند و متن را با اصلاح مکرر کل توالی به صورت موازی تولید می‌کنند، که اغلب به آنها مزیت قابل توجهی در سرعت استنتاج و توان عملیاتی برای پردازش دسته‌ای یا تولیدات طولانی‌تر می‌دهد. در حالی که یک مرحله نویززدایی در یک DLM می‌تواند از نظر محاسباتی سنگین‌تر از پیش‌بینی یک توکن توسط یک LLM باشد، توانایی تولید همزمان چندین توکن در چند مرحله به این معنی است که DLMها می‌توانند به زمان کمتری برای هر توکن خروجی (TPOT) دست یابند و آنها را به یک جایگزین نوظهور برای برنامه‌های حساس به تأخیر تبدیل می‌کند.

پرسش ۶۰: مفهوم پخش توکن (token streaming) در حین استنتاج را توضیح دهید.

پخش توکن در حین استنتاج LLM یک تکنیک بهینه‌سازی است که در آن خروجی مدل، معمولاً توکن پیش‌بینی شده بعدی، به محض تولید به کاربر ارسال می‌شود، به جای اینکه منتظر بمانیم تا کل پاسخ کامل شود. پخش توکن به طور قابل توجهی تأخیر درک شده را کاهش می‌دهد زیرا کاربر می‌تواند تقریباً بلافاصله شروع به خواندن خروجی کند و تعامل را بسیار سریع‌تر و پاسخگوتر احساس کند. پاسخ کامل به این ترتیب توکن به توکن "پخش" می‌شود تا زمانی که به توکن پایان توالی برسد.

پرسش ۶۱: رمزگشایی گمانه‌زانه (speculative decoding) چیست و چه زمانی از آن استفاده می‌کنید؟

رمزگشایی گمانه‌زانه با جفت کردن یک مدل "پیش‌نویس" کوچکتر و سریع‌تر با یک مدل "هدف" بزرگتر، استنتاج LLM را تسریع می‌کند. مدل پیش‌نویس به طور گمانه‌زانه چندین توکن آینده را از قبل تولید می‌کند، که مدل هدف سپس آنها را به صورت موازی تأیید می‌کند، آنهایی که با پیش‌بینی‌های خودش مطابقت دارند را می‌پذیرد و بقیه را اصلاح می‌کند. این رویکرد پیش‌نویس-و-تأیید، گلوگاه متوالی تولید توکن به توکن را کاهش می‌دهد، بهره‌وری GPU را بهبود می‌بخشد و تأخیر را بدون فدا کردن کیفیت خروجی کاهش می‌دهد. این روش به ویژه در برنامه‌های حساس به تأخیر مانند چت‌بات‌ها و تکمیل کد، که در آنها هم سرعت و هم دقت حیاتی است، مفید است.

پرسش ۶۲: چالش‌های اجرای استنتاج توزیع‌شده (distributed inference) در چندین GPU چیست؟

چالش‌های اصلی در اجرای استنتاج توزیع‌شده در چندین GPU شامل مدیریت حافظه، سربار ارتباطات، تعادل بار کاری و نیازهای منابع خاص هر فاز است.

1. **مدیریت حافظه** حیاتی است زیرا مدل‌های بزرگ اغلب در یک GPU جا نمی‌شوند و نیاز به پارتیشن‌بندی یا شاردینگ مدل در سراسر GPUها دارند.
2. **سربار ارتباطات** از همگام‌سازی پارامترها و داده‌های میانی بین GPUها ناشی می‌شود که می‌تواند تأخیر قابل توجهی اضافه کند.
3. **تعادل بار کاری** برای اطمینان از اینکه هیچ GPU واحدی به گلوگاه تبدیل نشود در حالی که بقیه کم‌استفاده هستند، مورد نیاز است و نیاز به استراتژی‌های موازی‌سازی مؤثر دارد.
4. در نهایت، فازهای مختلف استنتاج، مانند پیش‌پر کردن (compute-bound) و رمزگشایی (memory-bound)، به منابع GPU متفاوتی نیاز دارند که تخصیص و هماهنگی کارآمد منابع را پیچیده می‌کند. این چالش‌ها نیازمند هماهنگی و بهینه‌سازی دقیق برای به حداکثر رساندن توان عملیاتی و به حداقل رساندن تأخیر در سیستم‌های استنتاج توزیع‌شده چند-GPU است.

پرسش ۶۳: چگونه یک سیستم استنتاج LLM مقیاس‌پذیر برای برنامه‌های بی‌درنگ (real-time) طراحی می‌کنید؟

یک سیستم استنتاج LLM مقیاس‌پذیر برای برنامه‌های بی‌درنگ باید از شاردینگ مدل و فریم‌ورک‌های سرویس‌دهی توزیع‌شده مانند vLLM برای موازی‌سازی استنتاج در چندین GPU یا نود استفاده کند. سیستم باید دسته‌بندی درخواست‌ها، تعادل بار پویا و پردازش ناهمزمان را برای بهینه‌سازی بهره‌وری GPU و کاهش تأخیر پیاده‌سازی کند. کش کردن پرآمپت‌ها یا جاسازی‌های مکرر، پاسخ‌ها را بیشتر تسریع می‌کند، در حالی که سیاست‌های مقیاس‌بندی خودکار (autoscaling) کارایی منابع را در زمان اوج ترافیک تضمین می‌کنند. ترکیب کوانتیزه‌سازی و تقطیر (distillation) می‌تواند اندازه مدل را کاهش داده و عملکرد بی‌درنگ را بدون افت دقت عمده بهبود بخشد.

پرسش ۶۴: نقش فلش اتنشن (Flash Attention) در کاهش گلوگاه‌های حافظه در حین استنتاج را توضیح دهید.

فلش اتنشن با بهینه‌سازی نحوه مدیریت محاسبات توجه توسط مدل‌ها، نقش مهمی در کاهش گلوگاه‌های حافظه در حین استنتاج ایفا می‌کند. مکانیزم‌های توجه سنتی به دلیل انتقال مکرر داده‌ها بین حافظه با پهنای باند بالا (HBM) که کندتر است و SRAM روی تراشه که سریع‌تر اما کوچکتر است، سربار حافظه بالایی دارند و به طور مکرر کلیدها، پرس‌وجوها و مقادیر را برای هر مرحله بارگذاری و می‌نویسند. فلش اتنشن با انتقال محاسبه ماتریس بزرگ و میانی امتیازات توجه (Q.KT) از HBM کند به SRAM سریع‌تر روی تراشه، گلوگاه‌های حافظه را به طور قابل توجهی کاهش می‌دهد. این کار را با استفاده از تکنیکی به نام کاشی‌کاری (tiling) انجام می‌دهد، که در آن محاسبه توجه به بلوک‌های کوچکتر تقسیم شده و به صورت افزایشی محاسبه می‌شود، به این معنی که ماتریس توجه عظیم و کامل هرگز به صراحت در HBM کندتر ایجاد نمی‌شود. این رویکرد تأخیر دسترسی به حافظه را به شدت کاهش داده و سربار محاسباتی را کم می‌کند و استنتاج سریع‌تر و کارآمدتر از نظر حافظه را ممکن می‌سازد، که به ویژه برای توالی‌های ورودی طولانی مفید است.

پرسش ۶۵: دسته‌بندی پیوسته (continuous batching) چیست و چه تفاوتی با دسته‌بندی ایستا (static batching) دارد؟

دسته‌بندی ایستا شامل گروه‌بندی تعداد ثابتی از درخواست‌ها و پردازش همزمان آنهاست. عیب اصلی آن کارایی پایین است، زیرا تمام درخواست‌ها باید منتظر بمانند تا طولانی‌ترین توالی به پایان برسد، که منجر به زمان بیکاری GPU و افزایش تأخیر می‌شود. دسته‌بندی پیوسته یک تکنیک برتر و پویا است که در سطح تولید توکن عمل می‌کند و بلافاصله یک درخواست تکمیل شده را با یک درخواست جدید جایگزین می‌کند. این تفاوت کلیدی تضمین می‌کند که GPU به طور مداوم مورد استفاده قرار می‌گیرد و به طور چشمگیری توان عملیاتی کلی را افزایش داده و تأخیر را کاهش می‌دهد. دسته‌بندی ایستا اغلب در سناریوهای آفلاین که تأخیر اهمیت کمتری دارد ترجیح داده می‌شود، در حالی که دسته‌بندی پیوسته در برنامه‌های آنلاین و تعاملی می‌درخشد.

پرسش ۶۶: دقت ترکیبی (mixed precision) (مانند FP16) چیست و چرا در حین استنتاج استفاده می‌شود؟

دقت ترکیبی تکنیکی است که از ترکیبی از فرمت‌های عددی مختلف استفاده می‌کند، معمولاً از فرمت ممیز شناور ۱۶ بیتی (FP16) برای اکثر محاسبات، در کنار فرمت‌های با دقت بالاتر مانند FP32 در جایی که برای پایداری عددی لازم است، استفاده می‌کند. این تکنیک در حین استنتاج برای کاهش قابل توجه هم مصرف حافظه و هم زمان محاسبات استفاده می‌شود. نصف کردن عرض بیت، مصرف حافظه مدل را تقریباً به نصف کاهش می‌دهد، که این امکان را فراهم می‌کند که یا مدل‌های بزرگ‌تر روی GPU جا شوند یا اندازه دسته بزرگ‌تری استفاده شود. نکته مهم این است که سخت‌افزارهای مدرن مانند NVIDIA Tensor Cores می‌توانند عملیات FP16 را به طور قابل توجهی سریع‌تر اجرا کنند و در نتیجه توان عملیاتی کلی را با حداقل افت در دقت مدل افزایش دهند.

پرسش ۶۷: تفاوت بین سناریوهای استقرار استنتاج LLM آنلاین و آفلاین را بیان کرده و الزامات هر کدام را مورد بحث قرار دهید.

استنتاج LLM آنلاین شامل درخواست‌های بی‌درنگ و رو به کاربر است که معمولاً روی یک سرور ابری میزبانی می‌شود و به تأخیر کم و توان عملیاتی بالا برای مدیریت ترافیک غیرقابل پیش‌بینی و ارتباطات شبکه به طور کارآمد نیاز دارد. در مقابل، استنتاج LLM آفلاین با داده‌های از پیش جمع‌آوری شده به صورت دسته‌ای، معمولاً روی سخت‌افزار داخلی یا محلی، سروکار دارد، جایی که الزامات اصلی، توان عملیاتی بالا و پردازش حجم زیادی از داده‌ها در مقیاس است، با نیازهای تأخیر کمتر سختگیرانه. سناریوی آنلاین پاسخ‌های سریع فردی را در اولویت قرار می‌دهد، در حالی که سناریوی آفلاین بر پردازش داده‌ها در مقیاس عظیم و غیر بی‌درنگ تمرکز دارد.

پرسش ۶۸: بده‌بستان بین توان عملیاتی و تأخیر (throughput vs. latency) را در استنتاج LLM توضیح دهید.

تأخیر به مدت زمانی اشاره دارد که برای پردازش یک درخواست واحد طول می‌کشد - هر چه سریع‌تر پاسخ دهد، تأخیر کمتر است. این معمولاً با پردازش دسته‌های کوچکتر از داده‌ها در یک زمان به دست می‌آید. از سوی دیگر، توان عملیاتی تعداد درخواست‌هایی را که یک سیستم می‌تواند در هر ثانیه پردازش کند، اندازه‌گیری می‌کند، که این معیار زمانی بهبود می‌یابد که دسته‌های بزرگتر با هم پردازش شوند تا از GPU به طور کامل استفاده شود. با این حال، انجام این کار باعث می‌شود درخواست‌های فردی بیشتر منتظر بمانند و تأخیر افزایش یابد. از این رو، سیستم‌ها باید این معیارها را بر اساس کاربرد خود متعادل کنند - برنامه‌های تعاملی مانند چت‌بات‌ها بر تأخیر کم برای پاسخ‌های سریع تمرکز دارند، در حالی که سیستم‌های پردازش دسته‌ای به دنبال توان عملیاتی بالا برای به حداکثر رساندن کارایی هستند.

پرسش ۶۹: گلوگاه‌های مختلف در یک خط لوله استنتاج LLM معمولی هنگام اجرا روی یک GPU مدرن چیست؟

هنگام اجرای مدل‌های زبان بزرگ (LLMs) روی GPUهای مدرن، چندین گلوگاه کلیدی عملکرد را محدود می‌کنند. یک مسئله اصلی، اشباع پهنای باند حافظه است، جایی که مدل به طور مکرر به کش‌های بزرگ کلید-مقدار (KV) دسترسی پیدا می‌کند و حرکت داده‌ها را کند می‌کند. با رشد کش KV در طول تولید متن، حافظه GPU بیشتری مصرف می‌شود و اندازه‌های دسته کوچکتر را تحمیل کرده و فشار حافظه ایجاد می‌کند. گلوگاه‌های محاسباتی نیز در عملیات سنگین مانند ضرب ماتریس‌ها رخ می‌دهند، اگرچه اینها اغلب کمتر از تأخیرهای مربوط به حافظه حیاتی هستند. در سیستم‌های ترکیبی CPU-GPU، زمان‌بندی ناکارآمد وظایف می‌تواند هسته‌های GPU را کم‌استفاده باقی بگذارد، در حالی که تنظیمات چند-GPU با تأخیرهای ارتباطی اضافی مواجه هستند که مقیاس‌پذیری را کاهش می‌دهد. غلبه بر این چالش‌ها شامل تکنیک‌هایی مانند کش کردن، کوانتیزه‌سازی مدل، زمان‌بندی هوشمندانه‌تر و توزیع متعادل بار کاری برای بهره‌برداری کامل از قدرت GPU و به حداقل رساندن تأخیر است.

پرسش ۷۰: چگونه عملکرد استنتاج LLM را اندازه‌گیری می‌کنید؟

عملکرد استنتاج LLM عمدتاً با استفاده از معیارهای تأخیر و توان عملیاتی برای سنجش سرعت و کارایی مدل تحت بار اندازه‌گیری می‌شود. معیارهای کلیدی شامل زمان تا اولین توکن (TTFT) است که مدت زمان تولید اولین توکن توسط مدل پس از دریافت پرامپت را اندازه‌گیری می‌کند و بر تجربه کاربر در برنامه‌های بی‌درنگ تأثیر می‌گذارد. زمان برای هر توکن خروجی (TPOT) میانگین زمان تولید هر توکن بعدی را ارزیابی می‌کند و بر روانی جریان خروجی تأثیر می‌گذارد. تأخیر کلی، کل زمان از ارسال ورودی تا دریافت پاسخ کامل است. توان عملیاتی، معیار حیاتی دیگر، تعداد توکن‌ها یا درخواست‌هایی را که سیستم می‌تواند در واحد زمان پردازش کند، اندازه‌گیری می‌کند و مقیاس‌پذیری آن را نشان می‌دهد. این معیارها با هم به ارزیابی سرعت، پاسخگویی و مقیاس‌پذیری یک LLM در سناریوهای استقرار عملی کمک می‌کنند.

پرسش ۷۱: موتورهای استنتاج LLM مختلف موجود کدامند؟ کدام یک را ترجیح می‌دهید؟

برجسته‌ترین موتورهای استنتاج LLM امروزی عبارتند از:

- **vLLM**: که در توان عملیاتی بالا و کارایی حافظه از طریق PagedAttention و دسته‌بندی پیوسته برتری دارد.
- **NVIDIA TensorRT-LLM**: که با بهینه‌سازی خاص برای GPUهای NVIDIA با هسته‌های CUDA سفارشی، عملکرد اوج (کمترین تأخیر) را ارائه می‌دهد.
- **Hugging Face Text Generation Inference (TGI)**: یک راه‌حل قوی و آماده برای تولید که به خوبی با اکوسیستم Hugging Face یکپارچه شده است.
- موتورهای دیگر شامل **LMDeploy** و **llama.cpp** (برای دستگاه‌های CPU/لبه) هستند. ترجیح من به سمت vLLM به دلیل تعادل عالی بین توان عملیاتی بالا، سهولت استفاده (سازگاری با Hugging Face) و انعطاف‌پذیری خوب سخت‌افزاری آن است. این ویژگی‌ها vLLM را برای اکثر محیط‌های سرویس‌دهی مقیاس‌پذیر مبتنی بر ابر ایده‌آل می‌کند.

پرسش ۷۲: چالش‌های استنتاج LLM چیست؟

چالش‌های اصلی در استنتاج LLM عبارتند از تأخیر بالا، شدت محاسباتی، محدودیت‌های حافظه، محدودیت توکن، مشکلات دقت از جمله توهم (hallucinations) و نگرانی‌های مقیاس‌پذیری.

1. **تأخیر بالا** رخ می‌دهد زیرا LLMها خروجی را توکن به توکن تولید می‌کنند و در برنامه‌های بی‌درنگ تأخیر ایجاد می‌کنند.
2. **شدت محاسباتی** به این معنی است که اجرای LLMها به سخت‌افزار قدرتمند و گران‌قیمت نیاز دارد و منجر به هزینه‌های عملیاتی بالا می‌شود.
3. **محدودیت‌های حافظه** استقرار LLMها را بر روی دستگاه‌هایی با ظرفیت حافظه محدود، محدود می‌کند.
4. **محدودیت توکن** اندازه ورودی را محدود می‌کند و اغلب نیاز به کوتاه‌سازی دارد که می‌تواند درک زمینه را کاهش دهد.
5. **مشکلات دقت** مانند توهم می‌تواند قابلیت اطمینان خروجی را به خطر بیندازد.
6. **مقیاس‌پذیری** در مدیریت بسیاری از درخواست‌های همزمان بدون افت عملکرد یک چالش باقی می‌ماند.

پرسش ۷۳: گزینه‌های ممکن برای تسريع استنتاج LLM چیست؟

گزینه‌های ممکن برای تسريع استنتاج LLM عبارتند از:

- **کوانتیزه‌سازی (Quantization):** دقت عددی را برای کاهش محاسبات و مصرف حافظه کاهش می‌دهد و استنتاج را تسريع می‌کند.
- **هرس کردن (Pruning):** نوروها یا وزن‌های کمتر مهم را برای کوچکتر و سریعتر کردن مدل حذف می‌کند.
- **تقطیر دانش (Knowledge Distillation):** دانش را به یک مدل کوچکتر و کارآمدتر که سریعتر اجرا می‌شود، منتقل می‌کند.
- **کش کردن (KV Caching):** محاسبات توجه قبلی را برای تسريع تولید توکن ذخیره و دوباره استفاده می‌کند.
- **رمزگشایی گمانه‌زانه (Speculative Decoding):** از یک مدل پیش‌نویس کوچکتر برای تولید توکن‌های کاندیدا که توسط مدل کامل تأیید می‌شوند، استفاده می‌کند.
- **شتاب‌دهی سخت‌افزاری (Hardware Acceleration):** از GPUها، TPUها یا شتاب‌دهنده‌های سفارشی طراحی شده برای عملیات ماتریسی موازی استفاده می‌کند.

پرسش ۷۴: پرامپت‌نویسی زنجیره-فکر (Chain-of-Thought - CoT) چیست و چه زمانی بیشترین کاربرد را دارد؟

پرامپت‌نویسی زنجیره-فکر (CoT) تکنیکی است که در آن شما به یک LLM دستور می‌دهید تا قبل از رسیدن به پاسخ نهایی، فرآیند استدلال گام به گام را به صراحت نشان دهد. این فرآیند تفکر شبیه به انسان را تقلید می‌کند و مسائل پیچیده را به مراحل میانی قابل مدیریت تقسیم می‌کند. CoT برای وظایف استدلال پیچیده که نیاز به چندین مرحله دارند، مانند محاسبات ریاضی چند مرحله‌ای، استدلال نمادین و پاسخ به سؤالات عقل سلیم، بیشترین کاربرد را دارد. تکنیک پرامپت‌نویسی CoT دقت و توانایی LLM در مدیریت پیچیدگی را در مقایسه با پرامپت‌نویسی استاندارد به طور قابل توجهی بهبود می‌بخشد.

پرسش ۷۵: دلیل اثربخشی پرامپت‌نویسی زنجیره-فکر (CoT) را توضیح دهید.

قدرت پرامپت‌نویسی زنجیره-فکر (CoT) در این است که به مدل‌های زبان بزرگ کمک می‌کند تا بیشتر شبیه انسان‌ها فکر کنند - گام به گام. به جای پریدن مستقیم به یک پاسخ، CoT مدل را تشویق می‌کند تا با تقسیم مسئله به مراحل کوچکتر و منطقی، در مورد آن استدلال کند. این تفکر ساختاریافته، مدیریت وظایف مربوط به منطق، ریاضیات یا عقل سلیم را برای مدل آسان‌تر می‌کند و منجر به پاسخ‌های دقیق‌تر و قابل اعتمادتر می‌شود. به عبارت ساده، CoT به مدل کمی "زمان تفکر" اضافی می‌دهد قبل از اینکه در مورد پاسخ نهایی تصمیم بگیرد.

پرسش ۷۶: بده‌بستان‌های استفاده از پرامپت‌نویسی CoT را توضیح دهید.

پرامپت‌نویسی زنجیره-فکر (CoT) با تشویق مدل‌ها به تولید مراحل میانی قبل از ارائه پاسخ نهایی، دقت و تفسیرپذیری را بهبود می‌بخشد. با این حال، بده‌بستان‌هایی مانند افزایش تأخیر و مصرف توکن را به همراه دارد که سربارهای محاسباتی و هزینه‌ای را افزایش می‌دهد. علاوه بر این، CoT گاهی اوقات می‌تواند توهمات را تشدید کند اگر مسیر استدلال نادرست باشد. همچنین برای عملکرد بهینه، نیاز به طراحی دقیق پرامپت برای ایجاد تعادل بین جزئیات استدلال و اختصار دارد.

پرسش ۷۷: مهندسی پرامپت (prompt engineering) چیست و چرا برای LLM ها مهم است؟

مهندسی پرامپت فرآیند طراحی و اصلاح پرامپت‌های ورودی برای هدایت مؤثر LLM ها به سمت تولید پاسخ‌های دقیق، مرتبط و منسجم است. از آنجایی که LLM ها قصد کاربر را بر اساس نشانه‌های متنی تفسیر می‌کنند، پرامپت‌های خوب ساخته شده ابهام را کاهش داده و عملکرد را بهبود می‌بخشند. این امر به ویژه مهم است زیرا LLM ها به عبارت‌بندی، زمینه و مثال‌های ارائه شده در پرامپت حساس هستند. مهندسی پرامپت مؤثر کیفیت خروجی را بدون بازآموزی مدل افزایش می‌دهد و آن را به یک مهارت کلیدی در استفاده کارآمد از LLM ها برای کاربردهای متنوع تبدیل می‌کند.

پرسش ۷۸: تفاوت بین پرامپت‌نویسی بدون-شات (zero-shot) و چند-شات (few-shot) چیست؟

تفاوت اساسی در تعداد مثال‌های موجود در پرامپت نهفته است. در پرامپت‌نویسی بدون-شات، به مدل فقط دستورالعمل و داده‌های ورودی بدون مثال داده می‌شود. در اینجا مدل برای تولید خروجی صحیح کاملاً به دانش از پیش آموزش دیده خود متکی است. در مقابل، پرامپت‌نویسی چند-شات شامل تعداد کمی از مثال‌های ورودی-خروجی ("شات‌ها") در پرامپت است. این مثال‌ها LLM را راهنمایی می‌کنند تا فرمت، سبک یا وظیفه مورد نظر را بهتر درک کند و اغلب منجر به بهبود قابل توجهی در عملکرد می‌شود.

پرسش ۷۹: رویکردهای مختلف برای انتخاب مثال‌ها برای پرامپت‌نویسی چند-شات (few-shot) چیست؟

انتخاب مناسب مثال‌ها در پرامپت‌نویسی چند-شات با هدف به حداکثر رساندن اطلاع‌رسانی و ارتباط زمینه محدود ارائه شده به LLM است. یک رویکرد رایج، نمونه‌برداری تصادفی است، که در آن مثال‌ها به طور دلخواه از مجموعه داده انتخاب می‌شوند تا یک نمای کلی ارائه دهند. انتخاب مبتنی بر تنوع تضمین می‌کند که مثال‌ها طیف گسترده‌ای از سناریوهای ورودی و موارد خاص مرتبط با وظیفه را پوشش می‌دهند و از بیش‌برازش (overfitting) جلوگیری می‌کنند. در مقابل، انتخاب مبتنی بر شباهت (اغلب با استفاده از جاسازی‌های برداری) مثال‌هایی را بازیابی می‌کند که از نظر معنایی به پرس‌وجوی ورودی بیشترین شباهت را دارند و مستقیم‌ترین راهنمایی در-زمینه را ارائه می‌دهند.

پرسش ۸۰: چرا طول زمینه (context length) هنگام طراحی پرامپت‌ها برای LLM ها مهم است؟

طول زمینه بسیار مهم است زیرا حداکثر تعداد توکن‌هایی را که یک LLM می‌تواند هنگام تولید پاسخ در نظر بگیرد، تعیین می‌کند. اگر پرامپت شامل دستورالعمل‌ها، مثال‌ها و داده‌های ورودی از این حد فراتر رود، مدل ورودی را کوتاه می‌کند. این امر منجر به از دست رفتن اطلاعات حیاتی می‌شود که منجر به خروجی‌های ناقص، نامرتب یا نادرست می‌شود. بنابراین، درک و مدیریت طول زمینه برای طراحی پرامپت‌های مختصر و در عین حال جامع که از قابلیت‌های مدل به طور کامل بهره می‌برند، ضروری است.

پرسش ۸۱: پرامپت سیستمی (system prompt) چیست و چه تفاوتی با پرامپت کاربر (user prompt) دارد؟

پرامپت سیستمی دستوری است که به یک LLM داده می‌شود تا رفتار کلی، نقش یا سبک پاسخ آن را در طول یک مکالمه تعریف کند، مانند "شما یک دانشمند داده متخصص هستید." این پرامپت پایه‌ای را برای نحوه تفسیر و پاسخ مدل به ورودی‌ها تعیین می‌کند. در مقابل، پرامپت کاربر یک پرس‌وجو یا وظیفه مستقیم است که توسط کاربر در حین تعامل ارائه می‌شود، مانند "مفهوم جاسازی‌ها را توضیح دهید." به عبارت ساده، پرامپت سیستمی شخصیت و مرزهای مدل را تعریف می‌کند، در حالی که پرامپت کاربر پرس‌وجویی را که باید پاسخ داده شود یا وظیفه‌ای را که باید انجام شود، ارائه می‌دهد.

پرسش ۸۲: یادگیری در-زمینه (In-Context Learning - ICL) چیست و پرامپت‌نویسی چند-شات چه ارتباطی با آن دارد؟

یادگیری در-زمینه (ICL) یک توانایی قدرتمند مدل‌های زبان بزرگ است که به آنها اجازه می‌دهد وظایف را صرفاً با درک پرامپت ورودی انجام دهند. پرامپت‌نویسی چند-شات نوع خاصی از ICL است که در آن چند مثال برچسب‌دار به پرامپت اضافه می‌شود. مثال‌های اضافه شده به مدل نشان می‌دهند که وظیفه چگونه باید انجام شود و به آن کمک می‌کنند تا به ورودی‌های مشابه تعمیم دهد. ICL به LLMها اجازه می‌دهد تا به سرعت با داده‌های حداقلی و بدون بازآموزی با وظایف جدید سازگار شوند.

پرسش ۸۳: پرامپت‌نویسی خود-سازگاری (self-consistency prompting) چیست و چگونه استدلال را بهبود می‌بخشد؟

پرامپت‌نویسی خود-سازگاری تکنیکی در LLMها است که در آن مدل چندین مسیر استدلال یا پاسخ برای همان ورودی تولید می‌کند و سپس آنها را agregat می‌کند تا سازگارترین راه‌حل را انتخاب کند. به جای تکیه بر یک خروجی واحد، این روش توافق بین چندین تلاش استدلالی را در نظر می‌گیرد که احتمال خطاهای ناشی از مراحل استدلال جعلی یا مغرضانه را کاهش می‌دهد. این رویکرد با تقویت الگوهای صحیح و فیلتر کردن پاسخ‌های ناسازگار یا بعید، استدلال را بهبود می‌بخشد که منجر به خروجی‌های دقیق‌تر می‌شود.

پرسش ۸۴: چرا زمینه (context) در طراحی پرامپت‌ها مهم است؟

زمینه در طراحی پرامپت بسیار مهم است زیرا پس‌زمینه، محدودیت‌ها و نقش خاصی را که LLM باید اتخاذ کند، فراهم می‌کند. این به LLMها کمک می‌کند تا یک خروجی مرتبط و با کیفیت بالا تولید کنند. بدون زمینه کافی، LLM ممکن است قصد کاربر را اشتباه تفسیر کند و به دانش عمومی تکیه کند. این ممکن است منجر به تولید پاسخ‌های مبهم، عمومی یا نادرست شود که نیازهای خاص کاربر را برآورده نمی‌کند.

پرسش ۸۵: یک استراتژی برای کاهش توهم (hallucinations) از طریق طراحی پرامپت توصیف کنید.

یک استراتژی کلیدی برای کاهش توهم در LLM ها از طریق طراحی پرامپت، زمینه‌مند کردن (grounding) پاسخ با دستور دادن به مدل برای تکیه انحصاری بر زمینه ارائه شده است. این اغلب با گنجاندن یک دستورالعمل واضح مانند زیر به دست می‌آید: "به سوال زیر فقط با استفاده از سند ارائه شده پاسخ دهید. اگر سند حاوی پاسخ نیست، بگویید 'اطلاعات در سند موجود نیست.' از هیچ دانش خارجی استفاده نکنید." این کار فضای جستجوی مدل را به ورودی ارائه شده محدود می‌کند و احتمال تولید جزئیات ساختگی یا نادرست را به طور قابل توجهی کاهش می‌دهد.

پرسش ۸۶: چگونه یک پرامپت را ساختار می‌دهید تا اطمینان حاصل کنید که خروجی LLM در یک فرمت خاص، مانند JSON، باشد؟

برای اطمینان از اینکه خروجی LLM در یک فرمت خاص مانند JSON باشد، باید یک دستورالعمل صریح در پرامپت بگنجانید که به وضوح ساختار خروجی مورد نظر را بیان کند. به عنوان مثال، "لطفاً با یک شیء JSON معتبر پاسخ دهید." ارائه یک شما یا مثال از ساختار JSON مورد نیاز، شامل کلیدها و انواع داده‌های مورد انتظار برای هر مقدار، نیز مفید است. علاوه بر این، گنجاندن یک دستورالعمل صریح برای خروجی دادن فقط JSON و هیچ چیز دیگر به جلوگیری از گنجاندن متن توضیحی کمک می‌کند و منجر به یک خروجی تمیز و قابل تجزیه می‌شود.

پرسش ۸۷: هدف از پرامپت‌نویسی ReAct در عامل‌های هوش مصنوعی (AI Agents) را توضیح دهید.

پرامپت‌نویسی ReAct (استدلال و عمل) به عامل اجازه می‌دهد تا با امکان برنامه‌ریزی پویا، اجرای اقدامات خارجی (مانند استفاده از موتور جستجو یا ابزار) و اصلاح رویکرد خود بر اساس مشاهدات، وظایف پیچیده و چند مرحله‌ای را حل کند. این حلقه تفکر، عمل و مشاهده، استدلال مدل را در بازخورد دنیای واقعی زمینه‌مند نگه می‌دارد، که توهمات را کاهش می‌دهد و تصمیمات آن را دقیق‌تر، قابل تفسیرتر و قابل اعتمادتر می‌کند. با ترکیب استدلال منطقی با تعامل در دنیای واقعی، ReAct حل مسئله انعطاف‌پذیرتر، قابل اعتمادتر و شبیه به انسان را ممکن می‌سازد.

پرسش ۸۸: فازهای مختلف در توسعه LLM چیست؟

فرآیند توسعه معمول برای یک مدل زبان بزرگ (LLM) شامل سه فاز اصلی است:

- **پیش‌آموزشی (pre-training):** که در آن مدل درک زبان بنیادی را از مجموعه داده‌های عظیم و عمومی از طریق وظایف خودنظارتی مانند پیش‌بینی کلمه بعدی یاد می‌گیرد؛
- **تنظیم-دستورالعمل (instruction-tuning):** که در آن مدل پیش‌آموزش دیده بر روی مثال‌های متنوعی از دستورالعمل‌ها و خروجی‌های مطلوب مربوطه تنظیم دقیق می‌شود تا در پیروی از دستورالعمل‌ها و انجام وظایف خاص بهتر شود؛
- **تنظیم-همسویی (alignment tuning):** که رفتار مدل را برای همسویی با ارزش‌های انسانی، ترجیحات و استانداردهای ایمنی بیشتر اصلاح می‌کند و منجر به پاسخ‌های مفیدتر و بی‌ضررتر می‌شود.

پرسش ۸۹: انواع مختلف تنظیم دقیق (LLM fine-tuning) چیست؟

تنظیم دقیق، در زمینه LLM ها، یک اصطلاح گسترده است که می‌تواند به تنظیم دقیق دستورالعمل، تنظیم دقیق وظیفه-خاص یا تنظیم همسویی اشاره داشته باشد.

1. **تنظیم دقیق دستورالعمل (Instruction fine-tuning):** شامل تنظیم دقیق مدل بر روی مجموعه داده‌ای از جفت‌های با کیفیت (دستورالعمل، پاسخ) برای بهبود توانایی آن در پیروی از دستورالعمل‌ها و تعمیم در سراسر وظایف مختلف است.
2. **تنظیم دقیق وظیفه-خاص (Task-specific fine-tuning):** شامل تنظیم دقیق مدل بر روی مجموعه داده‌ای متناسب با یک برنامه پایین‌دستی خاص (مانند تحلیل احساسات، خلاصه‌سازی متن) برای به حداکثر رساندن عملکرد در آن وظیفه خاص است.
3. **تنظیم همسویی (Alignment Tuning):** شامل تنظیم دقیق مدل با استفاده از یادگیری تقویتی (RL) برای تنظیم رفتار مدل به گونه‌ای است که ایمن، مفید و همسو با ارزش‌ها و ترجیحات انسانی باشد.

پرسش ۹۰: تنظیم دستورالعمل (instruction tuning) چه نقشی در بهبود قابلیت استفاده یک LLM ایفا می‌کند؟

تنظیم دستورالعمل به طور قابل توجهی نحوه درک و پیروی یک مدل زبان بزرگ (LLM) از دستورات کاربر در پرامپت ورودی را بهبود می‌بخشد. در حالی که LLM های خام و پیش‌آموزش دیده که فقط برای پیش‌بینی کلمه بعدی آموزش دیده‌اند، در ادامه دادن متن خوب هستند. اما آنها اغلب در پیروی از دستورالعمل‌های صریح با مشکل مواجه می‌شوند. تنظیم دقیق دستورالعمل با آموزش مدل بر روی جفت‌های با کیفیت از پرامپت‌ها و پاسخ‌ها این مشکل را برطرف می‌کند. از طریق قرار گرفتن در معرض مثال‌های متنوع، مدل یاد می‌گیرد که دستورالعمل‌های کاربر را به درستی تفسیر کرده و وظایف داده شده را انجام دهد. به طور خلاصه، تنظیم دستورالعمل LLM ها را در درک پرامپت‌های کاربر و تولید خروجی‌های مطلوب بهتر می‌کند.

پرسش ۹۱: تنظیم همسویی (alignment tuning) چه نقشی در بهبود قابلیت استفاده یک LLM ایفا می‌کند؟

تنظیم همسویی با اطمینان از اینکه پاسخ‌های یک LLM با ارزش‌های انسانی و اصول اخلاقی همسو هستند، قابلیت استفاده آن را افزایش می‌دهد و تعاملات را ایمن‌تر و قابل ارتباط‌تر می‌کند. این فرآیند تضمین می‌کند که مدل نه تنها مفید است (به طور مؤثر به سؤالات پاسخ می‌دهد) بلکه بی‌ضرر نیز هست (از تولید محتوای سمی یا غیراخلاقی خودداری می‌کند). با تنظیم مدل بر روی یک سیگنال پاداش ترجیحی، همسویی باعث می‌شود خروجی LLM طبیعی‌تر، ایمن‌تر و قابل اعتمادتر به نظر برسد و اساساً تجربه کاربر را بهبود می‌بخشد.

پرسش ۹۲: چگونه از بیش‌برازش (overfitting) در حین تنظیم دقیق جلوگیری می‌کنید؟

برای جلوگیری از بیش‌برازش در حین تنظیم دقیق LLM، روش‌های اصلی شامل استفاده از یک مجموعه داده اعتبارسنجی (validation set) برای نظارت بر عملکرد و به کارگیری توقف زود هنگام (early stopping) زمانی که خطای اعتبارسنجی شروع به افزایش می‌کند، است که نشان می‌دهد مدل در حال حفظ کردن داده‌های آموزشی است. علاوه بر این، تکنیک‌هایی مانند تنظیم‌کنندگی (regularization) (مانند L2 یا dropout) می‌توانند برای جریمه کردن مدل‌های پیچیده اعمال شوند. در نهایت، اطمینان از اینکه مجموعه داده تنظیم دقیق متنوع و به اندازه کافی بزرگ نسبت به اندازه مدل است، از بیش‌برازش جلوگیری می‌کند و به مدل کمک می‌کند تا به داده‌های دیده‌نشده بهتر تعمیم یابد.

پرسش ۹۳: فراموشی فاجعه‌بار (catastrophic forgetting) چیست و چرا در تنظیم دقیق یک نگرانی محسوب می‌شود؟

فراموشی فاجعه‌بار از دست دادن قابلیت‌ها یا دانش قبلاً آموخته شده است زمانی که یک LLM بر روی داده‌های جدید و متمایز تنظیم دقیق می‌شود. این اتفاق می‌افتد زیرا تنظیم دقیق اغلب شامل به‌روزرسانی تمام یا بیشتر پارامترهای مدل است، که باعث می‌شود سیگنال آموزشی جدید وزن‌های مهم برای وظایف قدیمی را به شدت تغییر دهد. این یک نگرانی قابل توجه است زیرا سودمندی عمومی LLM پایه را به خطر می‌اندازد، به این معنی که مدل ممکن است در وظیفه جدید و تنظیم دقیق شده برتری داشته باشد اما قادر به انجام مجموعه اصلی و گسترده‌تری از وظایفی که در ابتدا برای آنها آموزش دیده بود، نباشد.

پرسش ۹۴: نقاط قوت و محدودیت‌های تنظیم دقیق کامل (full fine-tuning) چیست؟

تنظیم دقیق کامل شامل به‌روزرسانی تمام پارامترهای یک LLM پیش‌آموزش دیده است. نقطه قوت اصلی آن دستیابی به بالاترین عملکرد ممکن است زیرا مدل به طور کامل تطبیق داده می‌شود و به طور بالقوه منجر به نتایج پیشرفته می‌شود. با این حال، محدودیت‌های اصلی آن عبارتند از:

- از نظر محاسباتی گران و زمان‌بر بودن، نیاز به منابع GPU قابل توجه، فضای ذخیره‌سازی بزرگ برای نقاط بازرسی کامل مدل، و
- ریسک بالای فراموشی فاجعه‌بار، که در آن مدل دانش عمومی خود را که در طول پیش‌آموزشی به دست آورده است، از دست می‌دهد.

پرسش ۹۵: توضیح دهید که چگونه تنظیم دقیق با پارامتر کارآمد (PEFT) محدودیت‌های تنظیم دقیق کامل را برطرف می‌کند.

تنظیم دقیق با پارامتر کارآمد (PEFT) با به‌روزرسانی تنها زیرمجموعه کوچکی از پارامترهای مدل یا افزودن ماژول‌های سبک (معرفی تعداد حداقلی از پارامترهای جدید و قابل آموزش) در حالی که اکثریت وزن‌های مدل پیش‌آموزش دیده را ثابت نگه می‌دارد، محدودیت‌های تنظیم دقیق کامل را برطرف می‌کند. این رویکرد به طور قابل توجهی هزینه‌های محاسباتی، مصرف حافظه و نیازهای ذخیره‌سازی را در مقایسه با تنظیم دقیق کامل، که تمام پارامترها را به‌روزرسانی می‌کند و به منابع بالایی نیاز دارد، کاهش می‌دهد. PEFT همچنین با حفظ دانش اصلی پیش‌آموزش دیده، خطر فراموشی فاجعه‌بار و بیش‌برازش را که تنظیم دقیق کامل با آن مواجه است، کاهش می‌دهد. علاوه بر این، PEFT آموزش سریع‌تر و استقرار آسان‌تر بر روی دستگاه‌های لبه یا محیط‌های با منابع محدود را ممکن می‌سازد و آن را مقیاس‌پذیرتر و مقرون‌به‌صرفه‌تر می‌کند بدون اینکه عملکرد را فدا کند. این تعادل بین کارایی و اثربخشی، PEFT را به یک راه‌حل عملی برای تطبیق مدل‌های زبان بزرگ در حوزه‌ها و وظایف متعدد تبدیل می‌کند.

پرسش ۹۶: چه زمانی ممکن است مهندسی پرامپت بر تنظیم دقیق وظیفه-خاص ترجیح داده شود؟

مهندسی پرامپت به طور کلی بر تنظیم دقیق وظیفه-خاص ترجیح داده می‌شود زمانی که وظیفه پیچیده یا باز است و برای حل آن به دانش عمومی گسترده و توانایی‌های یادگیری در-زمینه LLM نیاز دارد. همچنین زمانی که داده برای تنظیم دقیق کمیاب است یا اگر نیاز به تکرار و آزمایش سریع وجود دارد، انتخاب بهتری است، زیرا اصلاح یک پرامپت به طور قابل توجهی سریع‌تر و از نظر منابع کارآمدتر از تنظیم دقیق مدل است. علاوه بر این، اگر یک LLM واحد باید انواع مختلفی از وظایف را مدیریت کند، مهندسی پرامپت ترجیح داده می‌شود، زیرا از ایجاد و استقرار یک مدل تنظیم دقیق شده جداگانه برای هر کدام جلوگیری می‌کند.

پرسش ۹۷: چه زمانی باید از تنظیم دقیق در مقابل RAG استفاده کنید؟

تنظیم دقیق بهترین گزینه است زمانی که می‌خواهید مدل دانش تخصصی حوزه را عمیقاً یاد بگیرد یا وظایف تخصصی را مدیریت کند که در آن دانش نسبتاً ثابت است و به طور مکرر تغییر نمی‌کند. از سوی دیگر، RAG (تولید تقویت‌شده با بازیابی) ایده‌آل است زمانی که نیاز دارید مدل به آخرین اطلاعات، اطلاعات اختصاصی یا اطلاعاتی که به طور مکرر تغییر می‌کنند، بدون بازآموزی دسترسی داشته باشد. این به مدل اجازه می‌دهد تا پاسخ‌های مبتنی بر واقعیت و قابلیت ردیابی منبع را از یک پایگاه دانش امن ارائه دهد.

پرسش ۹۸: محدودیت‌های استفاده از RAG را در مقایسه با تنظیم دقیق LLM توضیح دهید.

معایب اصلی استفاده از RAG در مقایسه با تنظیم دقیق در عملکرد و عدم تخصص عمیق آن نهفته است. از آنجایی که RAG یک مرحله بازبینی اضافی اضافه می‌کند، تأخیر استنتاج بیشتری را به همراه دارد و آن را برای موارد استفاده با تأخیر کم کمتر ایده‌آل می‌کند. در حالی که RAG در آوردن دانش خارجی مؤثر است، در واقع رفتار اصلی، سبک یا توانایی مدل در مدیریت استدلال پیچیده و تخصصی حوزه را تغییر نمی‌دهد. تنظیم دقیق با به‌روزرسانی وزن‌های مدل به تمام اینها دست می‌یابد. علاوه بر این، کیفیت خروجی RAG به شدت به دقت بازیاب (retriever) بستگی دارد، به این معنی که اگر اطلاعات نامرتب بازیابی شود، می‌تواند نتایج ضعیفی تولید کند.

پرسش ۹۹: محدودیت‌های استفاده از تنظیم دقیق LLM را در مقایسه با RAG توضیح دهید.

عیب اصلی تنظیم دقیق در مقایسه با RAG، دانش ایستا مدل است. پس از آموزش مدل، نمی‌تواند به اطلاعات جدید یا بی‌درنگ دسترسی پیدا کند مگر اینکه یک فرآیند بازآموزی گران و زمان‌بر را طی کند. تنظیم دقیق همچنین به منابع محاسباتی قابل توجه و تخصص ویژه برای آماده‌سازی داده‌ها و آموزش مدل نیاز دارد. علاوه بر این، مدل‌های تنظیم دقیق شده در معرض خطر "فراموشی فاجعه‌بار" هستند، که در آن یادگیری اطلاعات جدید باعث می‌شود بخشی از دانش عمومی اصلی خود را از دست بدهند. RAG از مشکل فراموشی فاجعه‌بار جلوگیری می‌کند، زیرا مدل پایه را بدون تغییر نگه می‌دارد.

پرسش ۱۰۰: چه زمانی باید تنظیم دقیق وظیفه-خاص را به مهندسی پرامپت ترجیح دهید؟

شما باید تنظیم دقیق وظیفه-خاص را انتخاب کنید زمانی که مهندسی پرامپت به تنهایی سطح عملکرد مورد نظر را ارائه نمی‌دهد. این اغلب در مورد وظایف پیچیده و تخصصی در زمینه‌هایی مانند حقوق یا پزشکی اتفاق می‌افتد، جایی که مدل پایه ممکن است فاقد دانش عمیق مورد نیاز حوزه باشد. تنظیم دقیق همچنین زمانی ایده‌آل است که شما به استنتاج با تأخیر کم و مقرون به صرفه در سیستم‌های تولیدی در مقیاس بزرگ نیاز دارید، زیرا یک مدل کوچکتر و بهینه‌تر نسبت به مدلی که به پرامپت‌های طولانی متکی است، تولید می‌کند. در نهایت، تنظیم دقیق کنترل بیشتری بر رفتار مدل ارائه می‌دهد و اطمینان از ثبات را آسان‌تر می‌کند، که پرامپت‌نویسی به تنهایی همیشه نمی‌تواند آن را تضمین کند.

پرسش ۱۰۱: LoRA چیست و در سطح بالا چگونه کار می‌کند؟

LoRA (تطبیق رتبه-پایین) یک تکنیک تنظیم دقیق با پارامتر کارآمد (PEFT) است که به شما امکان می‌دهد LLMها را با به‌روزرسانی تنها تعداد کمی از پارامترهای اضافی به جای تغییر تمام وزن‌های مدل، تنظیم دقیق کنید. در سطح بالا، LoRA با تزریق ماتریس‌های کوچک قابل آموزش و رتبه-پایین به لایه‌های خاصی از مدل (معمولاً لایه‌های توجه یا پیش‌خور) کار می‌کند. به جای به‌روزرسانی ماتریس وزن بزرگ W ، ماتریس W_S را ثابت نگه می‌دارد و یک تجزیه رتبه-پایین $\Delta W = AB$ را اضافه می‌کند، که در آن A و B ماتریس‌های بسیار کوچکتری هستند ($d \ll r$). در طول آموزش، فقط A و B یاد گرفته می‌شوند که به طور قابل توجهی هزینه‌های حافظه و محاسبات را کاهش می‌دهد. در زمان استنتاج، وزن‌های تطبیق داده شده به طور مؤثر با وزن‌های اصلی مدل ادغام می‌شوند، بنابراین مدل طوری رفتار می‌کند که گویی به طور کامل تنظیم دقیق شده است اما با پارامترهای بسیار کمتری که آموزش دیده‌اند. این امر LoRA را هم کارآمد (در ذخیره‌سازی و محاسبات) و هم ماژولار می‌کند.

پرسش ۱۰۲: مؤلفه کلیدی پشت اثربخشی تکنیک LoRA را توضیح دهید.

مؤلفه کلیدی پشت اثربخشی تکنیک LoRA در تجزیه رتبه-پایین به روزرسانی‌های وزن نهفته است، که اطلاعات وظیفه-خاص را در یک زیرفضای بسیار کوچکتر از فضای پارامتر کامل مدل ثبت می‌کند. با بیان تغییر وزن به عنوان حاصلضرب دو ماتریس کوچک، LoRA تنظیم دقیق کارآمد را با حداقل سربار حافظه و محاسباتی ممکن می‌سازد. این رویکرد از این مشاهده بهره می‌برد که مدل‌های زبان بزرگ پارامترهای اضافی دارند و بیشتر تطبیق‌ها را می‌توان در یک فرم با ابعاد پایین نمایش داد. در نتیجه، LoRA به عملکردی قابل مقایسه با تنظیم دقیق کامل دست می‌یابد در حالی که تنها بخش کوچکی از پارامترهای مدل را آموزش می‌دهد.

پرسش ۱۰۳: QLoRA چیست و چه تفاوتی با LoRA دارد؟

QLoRA (تطبیق رتبه-پایین کوانتیزه‌شده) توسعه‌ای از LoRA است که تنظیم دقیق مدل‌های زبان بزرگ را با ترکیب تطبیق رتبه-پایین با کوانتیزه‌سازی کارآمدتر می‌کند. در حالی که LoRA بیشتر وزن‌های مدل را ثابت نگه می‌دارد و ماتریس‌های کوچک رتبه-پایین را برای کاهش هزینه‌های حافظه و محاسبات آموزش می‌دهد، QLoRA یک گام فراتر می‌رود و وزن‌های مدل را در حین تنظیم دقیق به دقت ۴ بیتی کوانتیزه می‌کند. این کار به طور چشمگیری نیاز به حافظه GPU را بدون افت عملکرد قابل توجه کاهش می‌دهد و امکان تنظیم دقیق مدل‌های بسیار بزرگ را روی سخت‌افزار مصرفی فراهم می‌کند.

پرسش ۱۰۴: چه زمانی از QLoRA به جای LoRA استاندارد استفاده می‌کنید؟

QLoRA بر LoRA استاندارد ترجیح داده می‌شود زمانی که مدل‌های زبان بسیار بزرگ را روی منابع سخت‌افزاری محدود، مانند یک GPU با حافظه محدود، تنظیم دقیق می‌کنید. کوانتیزه‌سازی ۴ بیتی QLoRA به طور قابل توجهی مصرف حافظه را کاهش می‌دهد در حالی که عملکرد مدل را حفظ می‌کند و آن را برای تنظیم دقیق با منابع کارآمد ایده‌آل می‌سازد. این روش به ویژه هنگام کار با مدل‌هایی مانند LLaMA که در غیر این صورت از محدودیت‌های GPU فراتر می‌روند، مفید است. در مقابل، LoRA استاندارد زمانی کافی است که ظرفیت سخت‌افزار یک محدودیت عمده نباشد.

پرسش ۱۰۵: چگونه تنظیم دقیق LLM را روی سخت‌افزار مصرفی با حافظه GPU محدود مدیریت می‌کنید؟

هنگام تنظیم دقیق یک LLM روی سخت‌افزار مصرفی با حافظه GPU محدود، می‌توان از تکنیک‌هایی مانند LoRA (تطبیق رتبه-پایین) یا QLoRA برای کاهش مصرف حافظه با آموزش تنها زیرمجموعه کوچکی از پارامترها استفاده کرد. علاوه بر این، استفاده از انباشت گرادیان (gradient accumulation)، آموزش با دقت ترکیبی (mixed precision training)، اندازه‌های دسته کوچکتر و طول‌های توالی کوچکتر به مدیریت محدودیت‌های GPU کمک می‌کند. این رویکردها سربار حافظه و هزینه محاسباتی را به حداقل می‌رسانند و تنظیم دقیق LLM را روی دستگاه‌های با منابع محدود ممکن می‌سازند.

پرسش ۱۰۶: روش‌های مختلف همسویی ترجیحات (preference alignment) و بده‌بستان‌های آنها را توضیح دهید.

روش‌های مختلف همسویی ترجیحات در LLM شامل یادگیری تقویتی از بازخورد انسانی (RLHF)، بهینه‌سازی ترجیحات مستقیم (DPO)، بهینه‌سازی ترجیحات نسبت شانس (ORPO) و بهینه‌سازی کانمن-تورسکی (KTO) است.

1. **RLHF** از یک مدل پاداش آموزش دیده بر روی نمونه‌های برچسب‌گذاری شده توسط انسان برای هدایت رفتار مدل استفاده می‌کند اما از نظر محاسباتی گران و پیچیده است.
2. **DPO** این فرآیند را با بهینه‌سازی مستقیم مدل بر روی داده‌های ترجیحی بدون یک مدل پاداش جداگانه ساده می‌کند، که کارایی و پایداری را ارائه می‌دهد اما به شدت به کیفیت داده‌ها وابسته است.
3. **ORPO** اهداف وظیفه و همسویی ترجیحات را در یک تابع زیان ترکیب می‌کند و کارایی آموزش را بهبود می‌بخشد اما پیچیدگی پیاده‌سازی را افزایش می‌دهد.
4. **KTO** از برچسب‌های دوتایی خوب/بد استفاده می‌کند، در برابر داده‌های نویزدار مقاوم است و برچسب‌گذاری آن ساده است، اما ممکن است برای همسویی ظریف، فاقد جزئیات باشد. بده‌بستان‌ها حول محور پیچیدگی، منابع محاسباتی، نیازهای داده و دقت همسویی می‌چرخند و روش بهینه بر اساس موارد استفاده خاص و محدودیت‌های منابع انتخاب می‌شود.

پرسش ۱۰۷: انباشت گرادیان (gradient accumulation) چیست و چگونه به تنظیم دقیق مدل‌های بزرگ کمک می‌کند؟

انباشت گرادیان تکنیکی است که در تنظیم دقیق مدل‌های زبان بزرگ (LLMs) استفاده می‌شود و با شبیه‌سازی اندازه‌های دسته بزرگتر به مدیریت محدودیت‌های حافظه GPU کمک می‌کند. به جای به‌روزرسانی وزن‌های مدل پس از پردازش هر مینی-دسته، گرادیان‌ها در طول چندین دسته کوچکتر انباشته می‌شوند و پارامترهای مدل یک بار پس از انباشت به‌روزرسانی می‌شوند. این رویکرد آموزش با اندازه‌های دسته مؤثر بزرگ را حتی روی سخت‌افزار با حافظه محدود ممکن می‌سازد و پایداری و عملکرد فرآیند تنظیم دقیق را بهبود می‌بخشد. این امکان تنظیم دقیق LLM‌ها را روی GPU‌های کمتر قدرتمند فراهم می‌کند و هزینه سخت‌افزار را کاهش می‌دهد در حالی که کیفیت آموزش را حفظ می‌کند.

پرسش ۱۰۸: گزینه‌های ممکن برای تسریع تنظیم دقیق LLM چیست؟

برای تسریع تنظیم دقیق LLM، می‌توان از چندین استراتژی بهینه‌سازی استفاده کرد. تکنیک‌هایی مانند LoRA (تطبیق رتبه-پایین) یا QLoRA (کوانتیزه‌شده) تعداد پارامترهای قابل آموزش را کاهش می‌دهند و به طور قابل توجهی مصرف حافظه GPU و زمان آموزش را کاهش می‌دهند. علاوه بر این، آموزش با دقت ترکیبی (FP16/BF16) برای محاسبات سریع‌تر، انباشت گرادیان برای شبیه‌سازی اندازه‌های دسته بزرگتر و آموزش توزیع‌شده در چندین GPU به همگرایی سریع‌تر و زمان آموزش کوتاه‌تر دست می‌یابند. این رویکردها هم هزینه محاسباتی و هم زمان تنظیم دقیق را به شدت کاهش می‌دهند.

پرسش ۱۰۹: هدف پیش‌آموزشی (pretraining objective) مورد استفاده در پیش‌آموزشی LLM را توضیح دهید.

هدف پیش‌آموزشی مدل‌های زبان بزرگ، پیش‌بینی توکن بعدی است که به عنوان مدل‌سازی زبان علی (causal language modeling) نیز شناخته می‌شود. در این تنظیم، مدل یاد می‌گیرد که توکن بعدی را در یک توالی با توجه به تمام توکن‌های قبلی پیش‌بینی کند. با به حداقل رساندن تفاوت بین پیش‌بینی‌های خود و توکن‌های واقعی بعدی در میلیاردها مثال، مدل به تدریج گرامر، معناشناسی و روابط متنی را یاد می‌گیرد. این هدف به LLM‌ها امکان می‌دهد تا متن منسجم و مرتبط با زمینه تولید کنند و طیف گسترده‌ای از وظایف پایین‌دستی را از طریق پرامپت‌نویسی یا تنظیم دقیق انجام دهند.

پرسش ۱۱۰: تفاوت بین مدل‌سازی زبان علی (causal language modeling) و مدل‌سازی زبان ماسک‌دار (masked language modeling) چیست؟

مدل‌سازی زبان علی (CLM) یک رویکرد خودبازگشتی است که در آن مدل توکن بعدی را در یک توالی فقط بر اساس توکن‌های قبلی پیش‌بینی می‌کند. در مقابل، مدل‌سازی زبان ماسک‌دار (MLM) یک رویکرد خودرمزگذار (autoencoding) است که در آن مدل توکن‌های عمداً ماسک‌دار (حذف شده) را با استفاده از زمینه دوطرفه پیش‌بینی می‌کند، یعنی هم توکن‌های گذشته و هم آینده را در توالی در نظر می‌گیرد.

پرسش ۱۱۱: LLMها چگونه با کلمات خارج از واژگان (OOV) برخورد می‌کنند؟

LLMها با استفاده از روش‌های توکن‌سازی زیرکلمه‌ای مانند WordPiece، Byte Pair Encoding (BPE) یا SentencePiece با کلمات خارج از واژگان (OOV) برخورد می‌کنند. این تکنیک‌ها کلمات نادر یا دیده‌نشده را به واحدهای زیرکلمه‌ای یا کاراکترهای کوچکتر و شناخته‌شده تقسیم می‌کنند. این به مدل اجازه می‌دهد تا کلمات جدید را از توکن‌های موجود در واژگان نمایش دهد و درک کند. به عنوان مثال، کلمه "unhappiness" ممکن است به "happy"، "un"، و "ness" تقسیم شود. این رویکرد مشکل OOV را کاهش می‌دهد و تعمیم‌پذیری به واژگان دیده‌نشده را بهبود می‌بخشد.

پرسش ۱۱۲: در زمینه پیش‌آموزشی LLM، قانون مقیاس‌پذیری (scaling law) چیست؟

در زمینه پیش‌آموزشی LLM، قوانین مقیاس‌پذیری رابطه قابل پیش‌بینی بین عملکرد یک مدل و عوامل کلیدی آن—مانند اندازه مدل (تعداد پارامترها)، اندازه مجموعه داده و منابع محاسباتی—را توصیف می‌کنند. مطالعات تجربی نشان می‌دهند که با افزایش این عوامل، عملکرد مدل از یک روند قانون-توان (power-law) پیروی می‌کند تا زمانی که بازده کاهشی ظاهر شود. این قانون با پیش‌بینی تعادل بهینه داده، پارامترها و محاسبات مورد نیاز برای دستیابی به یک سطح عملکرد هدف، یک راهنمای حیاتی برای طراحی و تخصیص کارآمد منابع برای آموزش مدل در مقیاس بزرگ فراهم می‌کند.

پرسش ۱۱۳: مفهوم معماری ترکیب-متخصصان (Mixture-of-Experts - MoE) و نقش آن در پیش‌آموزشی LLM را توضیح دهید.

معماری ترکیب-متخصصان (MoE) با جایگزین کردن لایه‌های استاندارد پیش‌خور با مجموعه‌ای از شبکه‌های "متخصص" تخصصی، کارایی و ظرفیت پیش‌آموزشی مدل زبان بزرگ (LLM) را به طور قابل توجهی بهبود می‌بخشد. یک "مسیریاب" یا "شبکه دروازه‌بندی" یاد می‌گیرد که برای هر توکن ورودی، زیرمجموعه کوچکی از این متخصصان را به طور انتخابی فعال کند. این به مدل اجازه می‌دهد تا (i) تعداد کل پارامترهای خود و در نتیجه ظرفیت دانش خود را به طور چشمگیری افزایش دهد و (ii) هزینه محاسباتی پایینی را در حین استنتاج حفظ کند، زیرا تنها بخشی از پارامترها برای هر ورودی معین استفاده می‌شود. این پراکندگی به مدل‌هایی با میلیاردها پارامتر اجازه می‌دهد تا به طور کارآمدتر آموزش دیده و اجرا شوند و مقیاس‌پذیری LLMها را به اندازه‌های بی‌سابقه‌ای تسهیل می‌کند.

پرسش ۱۱۴: موازی‌سازی مدل (model parallelism) چیست و چگونه در پیش‌آموزشی LLM استفاده می‌شود؟

موازی‌سازی مدل تکنیکی است که برای آموزش مدل‌های زبان بزرگی که برای جا شدن در یک GPU بیش از حد بزرگ هستند، با تقسیم پارامترهای مدل در چندین دستگاه استفاده می‌شود. به جای اینکه هر GPU یک کپی کامل از مدل را نگه دارد، GPUهای مختلف لایه‌های مختلف یا بخش‌هایی از همان لایه را مدیریت می‌کنند. در طول پاس‌های پیشرو و پسرو، فعال‌سازی‌ها و گرادیان‌ها برای تکمیل محاسبات بین GPUها منتقل می‌شوند. این امکان استفاده کارآمد از سخت‌افزار را برای مدل‌های عظیم فراهم می‌کند. با این حال، نیاز به هماهنگی دقیق برای به حداقل رساندن سرشار ارتباطات و تأخیر دارد.

پرسش ۱۱۵: اهمیت یادگیری خودنظارتی (self-supervised learning) در پیش‌آموزشی LLM چیست؟

یادگیری خودنظارتی (SSL) برای پیش‌آموزشی LLM حیاتی است زیرا به مدل‌ها اجازه می‌دهد تا نمایش‌های زبانی غنی و با کاربرد عمومی را از مقادیر عظیمی از داده‌های متنی بدون برچسب یاد بگیرند. این کار با ایجاد یک وظیفه جایگزین، مانند پیش‌بینی کلمه بعدی، عمل می‌کند که در آن "برچسب‌ها" به طور خودکار از خود ورودی مشتق می‌شوند. این امر نیاز به حاشیه‌نویسی گران‌قیمت انسانی را از بین می‌برد. با پیش‌بینی توکن بعدی در یک توالی، مدل نحو، معناشناسی و دانش جهانی را از مجموعه داده‌های عظیم بدون برچسب یاد می‌گیرد.